

Chapter 8

Signature Schemes

Stefan Dziembowski

www.crypto.edu.pl/Dziembowski

University of Warsaw



Plan



1. The definition of secure signature schemes
2. Signatures based on RSA, “hash-and-sign”, “full-domain-hash”
3. Constructions based on discrete log
 - a) identification schemes
 - b) Schnorr signatures
 - c) DSA signatures
4. Theoretical constructions

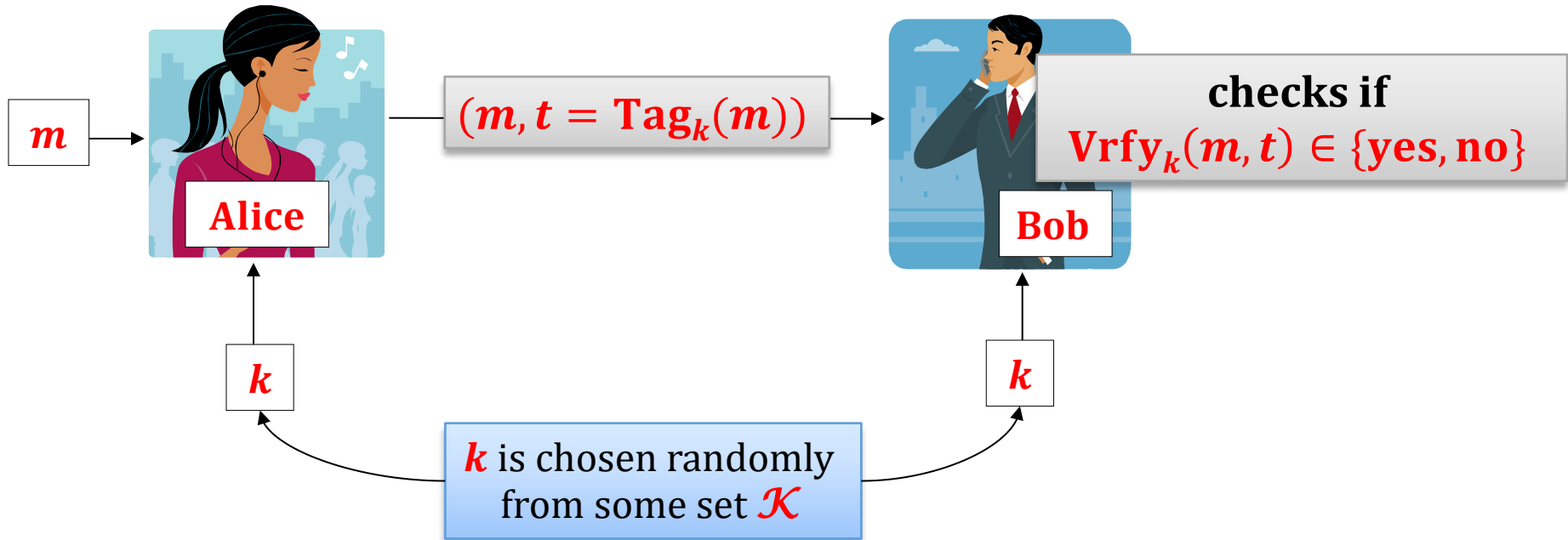
Signature schemes

digital signature schemes



MACs in the public-key setting

Message Authentication Codes

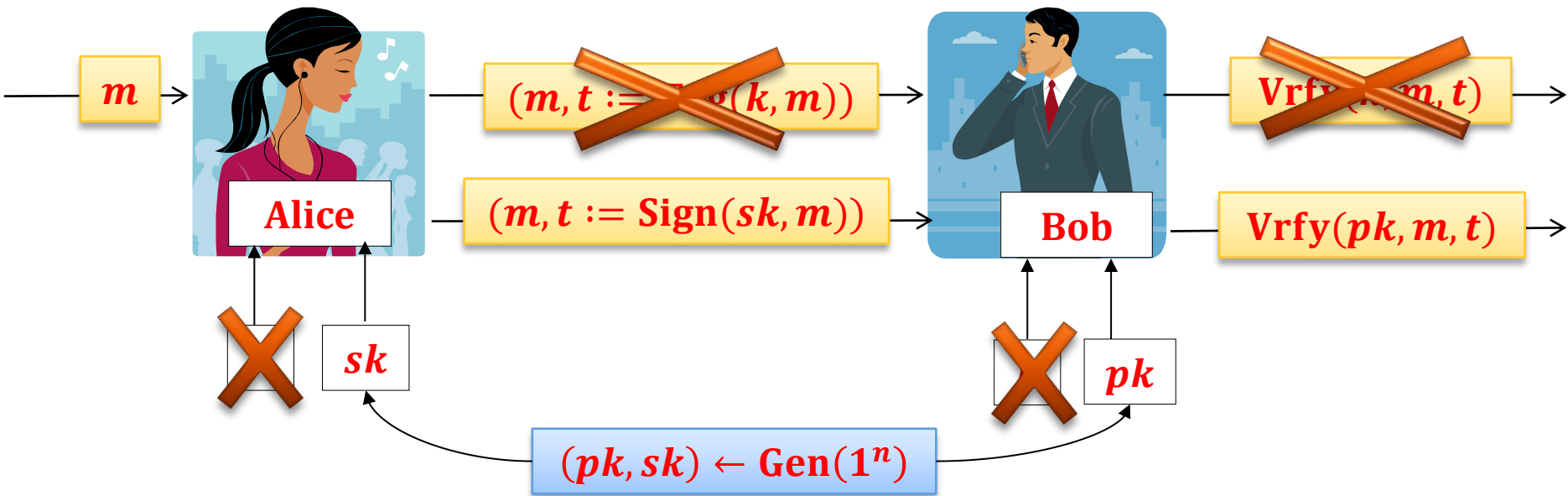


Signatures

- sk is used for **computing a tag**,
- pk is used for **verifying correctness of the tag**.

this will be called
“signatures”

Sign – the signing algorithm



Digital Signature Schemes

A **digital signature scheme** is a tuple $(\text{Gen}, \text{Sign}, \text{Vrfy})$ of polynomial-time algorithms, such that:

- the **key-generation** algorithm **Gen** takes as input a security parameter 1^n and outputs a pair (pk, sk) ,
- the **signing** algorithm **Sign** takes as input a key sk and a message $m \in \{0, 1\}^*$ and outputs a signature σ ,
- the **verification** algorithm **Vrfy** takes as input a key pk , a message m and a signature σ , and outputs a bit $b \in \{\text{yes}, \text{no}\}$.

If $\text{Vrfy}_{pk}(m, \sigma) = \text{yes}$ then we say that σ is a **valid signature on the message m** .

Correctness

We require that it always holds that:

$P(\text{Vrfy}_{pk}(m, \text{Sign}_{sk}(m)) \neq \text{yes})$ is negligible in n

What remains is to define **security**.

How to define security?

We have to assume that the adversary can see some pairs

$$(\mathbf{m}_1, \sigma_1), \dots, (\mathbf{m}_t, \sigma_t)$$

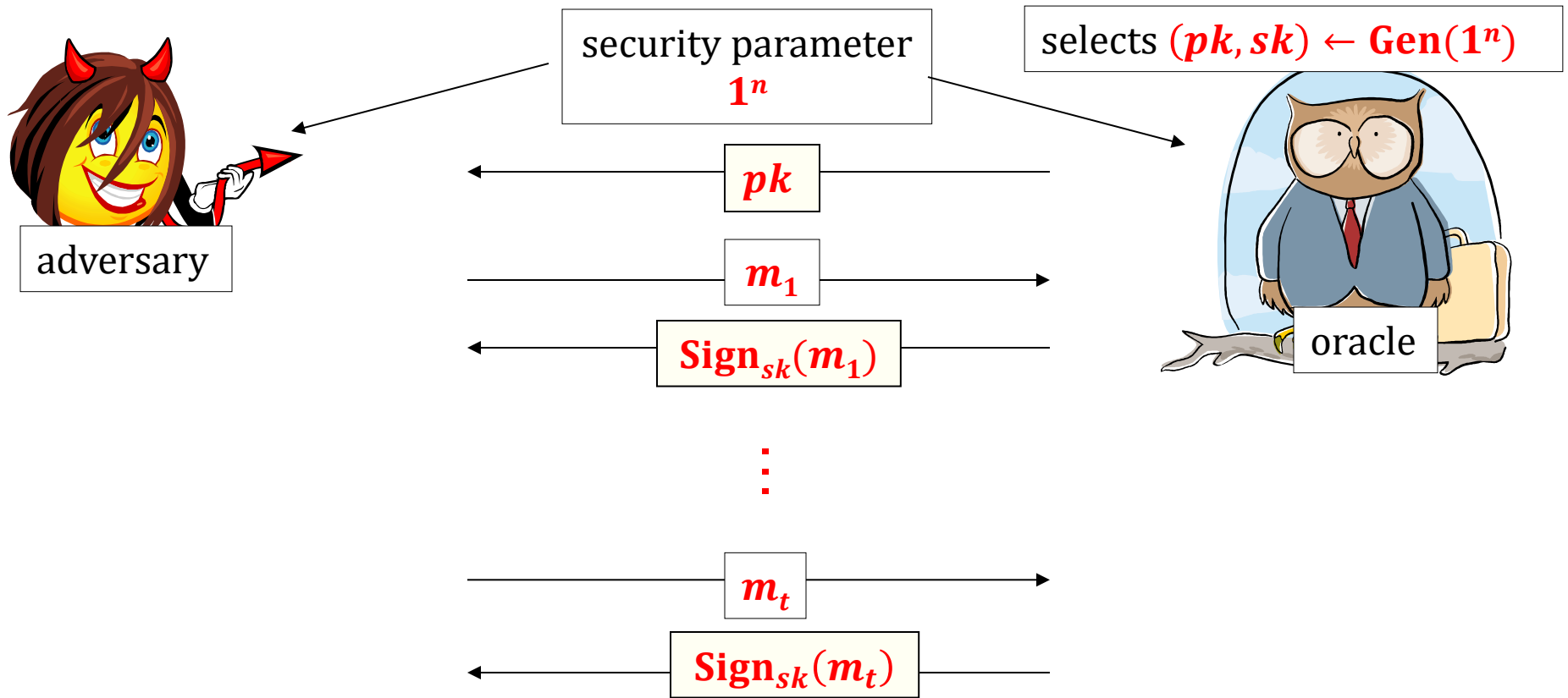
As in the case of MACs, we need to specify:

1. how the messages $\mathbf{m}_1, \dots, \mathbf{m}_t$ are chosen,
2. what is the goal of the adversary.

Good tradition: be as pessimistic as possible!

Therefore we assume that:

1. The adversary is allowed to chose $\mathbf{m}_1, \dots, \mathbf{m}_t$.
2. The **goal of the adversary** is to produce a valid signature on some $\mathbf{m}'$ such that $\mathbf{m}' \notin \{\mathbf{m}_1, \dots, \mathbf{m}_t\}$.



We say that the adversary **breaks the signature scheme** if at the end she outputs (m', σ') such that

1. $\text{Vrfy}_{pk}(m', \sigma') = \text{yes}$
2. $m' \notin \{m_1, \dots, m_t\}$.

The security definition

sometimes we just say: **unforgeable** (if the context is clear)

We say that **(Gen, Sign, Vrfy)** is **existentially unforgeable under an adaptive chosen-message attack** if



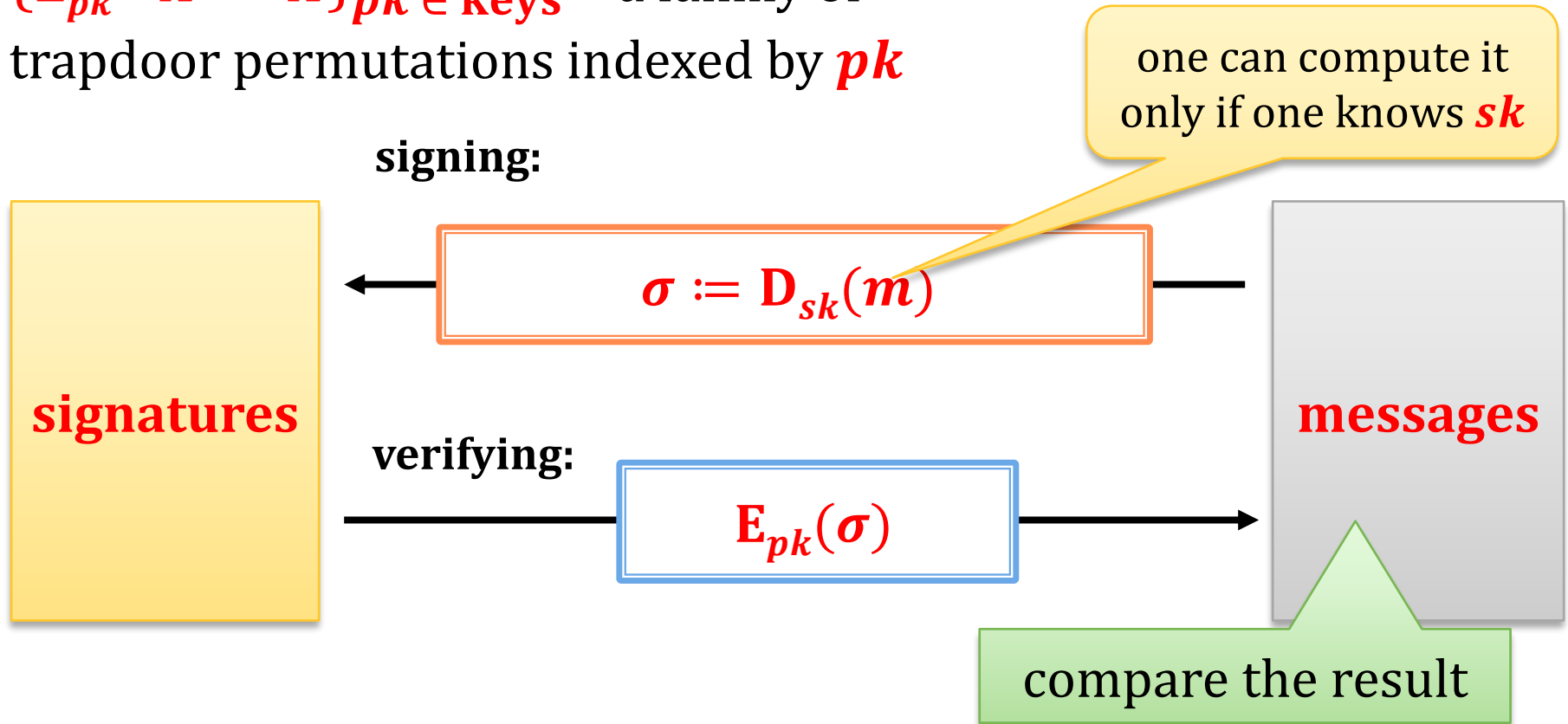
$P(A \text{ breaks it})$ is negligible (in **n**)

polynomial-time
adversary **A**

How to construct signature schemes?

Remember this idea?

$\{E_{pk} : X \rightarrow X\}_{pk \in \text{keys}}$ - a family of trapdoor permutations indexed by pk



We said: In general it's not that simple.

In general it's not that simple

Not every trapdoor permutation is OK.

example: the **RSA** function

There exist **other ways** to create signature schemes.

One can even construct a signature scheme **from any one-way function**.

(this is a theoretical construction)

Plan

1. The definition of secure signature schemes



2. Signatures based on RSA, “hash-and-sign”, “full-domain-hash”

3. Constructions based on discrete log

- a) identification schemes

- b) Schnorr signatures

- c) DSA signatures

4. Theoretical constructions

The “handbook RSA signatures”

$N = pq$, such that p and q are random primes,
and $|p| = |q|$

e – random such that $e \perp (p - 1)(q - 1)$

d – such that $ed = 1 \pmod{(p - 1)(q - 1)}$

messages and signatures: Z_N

- $\sigma := \text{Sign}_{N,d}(m) = m^d \bmod N$
- $\text{Vrfy}_{N,e}(m, \sigma) = \text{output yes iff } \sigma^e \bmod N = m$

Problems with the “handbook RSA” [1/2]

“no-message attack”:

The adversary can forge a signature on a “random” message m .

Given the public key (N, e) :

he just selects a random $\sigma \leftarrow \mathbb{Z}_N$ and computes

$$m := \sigma^e \bmod N.$$

Trivially, σ is a valid signature on m .

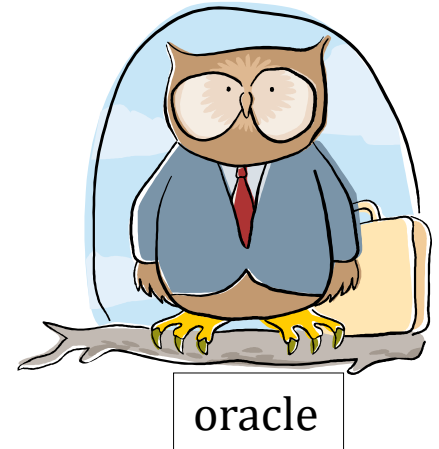
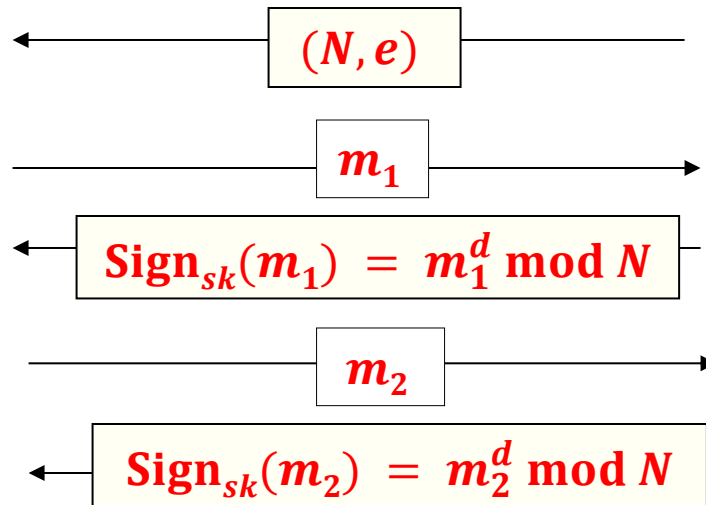
Problems with the “handbook RSA” [2/2]

How to forge a signature on an arbitrary message m ?
Use the homomorphic properties of RSA.



chooses:

1. any $m_1 \notin \{m, 1\}$
2. $m_2 := m/m_1 \bmod N$



computes (**mod N**):

$$\begin{aligned} & m_1^d \cdot m_2^d \\ = & (m_1 \cdot m_2)^d \\ = & m^d \end{aligned}$$

this is a valid signature on m

Is it a problem?

In many applications – probably not.

But we would like to have schemes that are **not**
application-dependent...

Solution

Before computing the **RSA function** – apply some function **H**.

N = **pq**, such that **p** and **q** are random primes,
and **|p|** = **|q|**

e – random such that **e** $\perp$ (**p** – 1)(**q** – 1)

d – such that **ed** = 1 (mod (**p** – 1)(**q** – 1))

messages and **signatures**: **Z_N**

- **$\sigma := \text{Sign}_{N,d}(m) = (\mathbf{H}(m))^d \bmod N$**
- **$\text{Vrfy}_{N,e}(m, \sigma) = \text{output yes iff } \sigma^e \bmod N = \mathbf{H}(m)$**

How to choose such H ?

A minimal requirement:

it should be collision-resistant.

(because if the adversary can find two messages m, m'
such that

$$H(m) = H(m')$$

then he can forge a signature on m' by asking the oracle
for a signature on m)

A typical choice of H

Usually H is one of the popular **hash functions**.

Additional advantage:

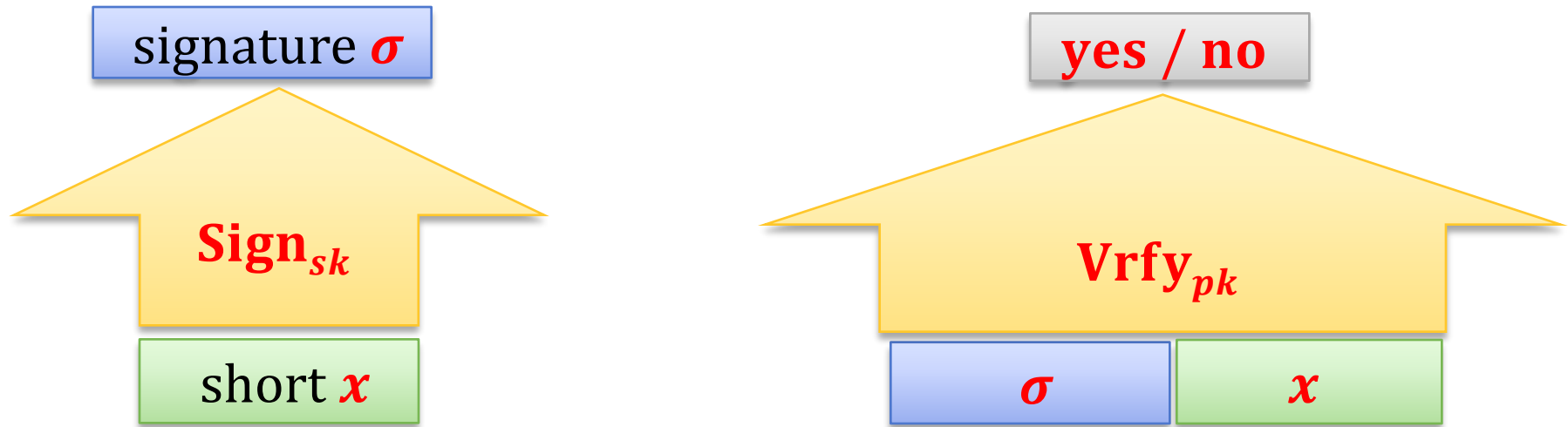
We can sign **very long messages** keeping the modulus N small (it's much more efficient!) – we will come back to it later.

It is called a

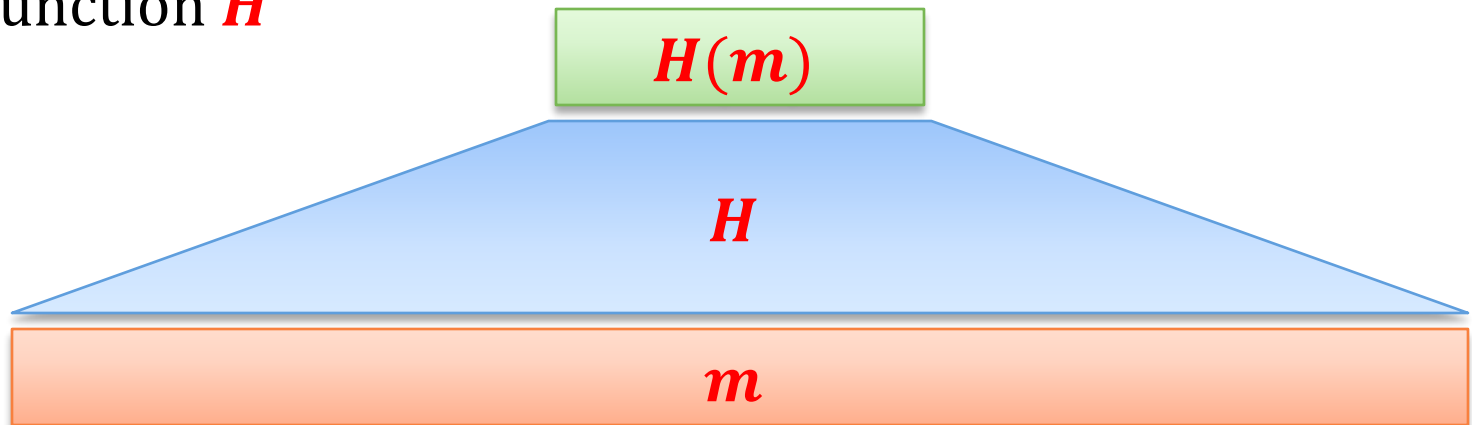
hash-and-sign paradigm.

Hash-and-Sign [1/4]

1. **(Gen, Sign, Vrfy)** – a signature scheme “for short messages”

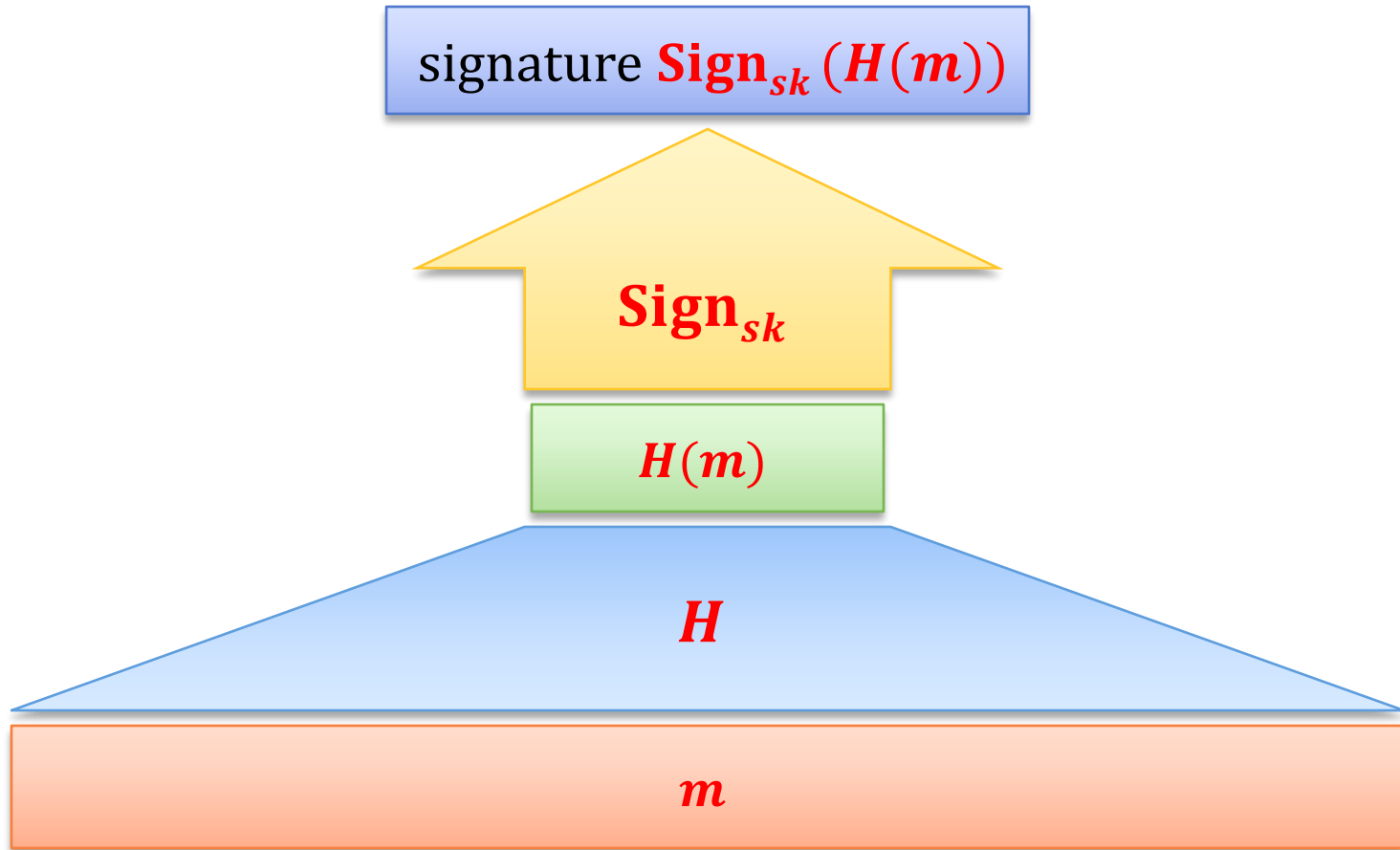


2. a hash function H



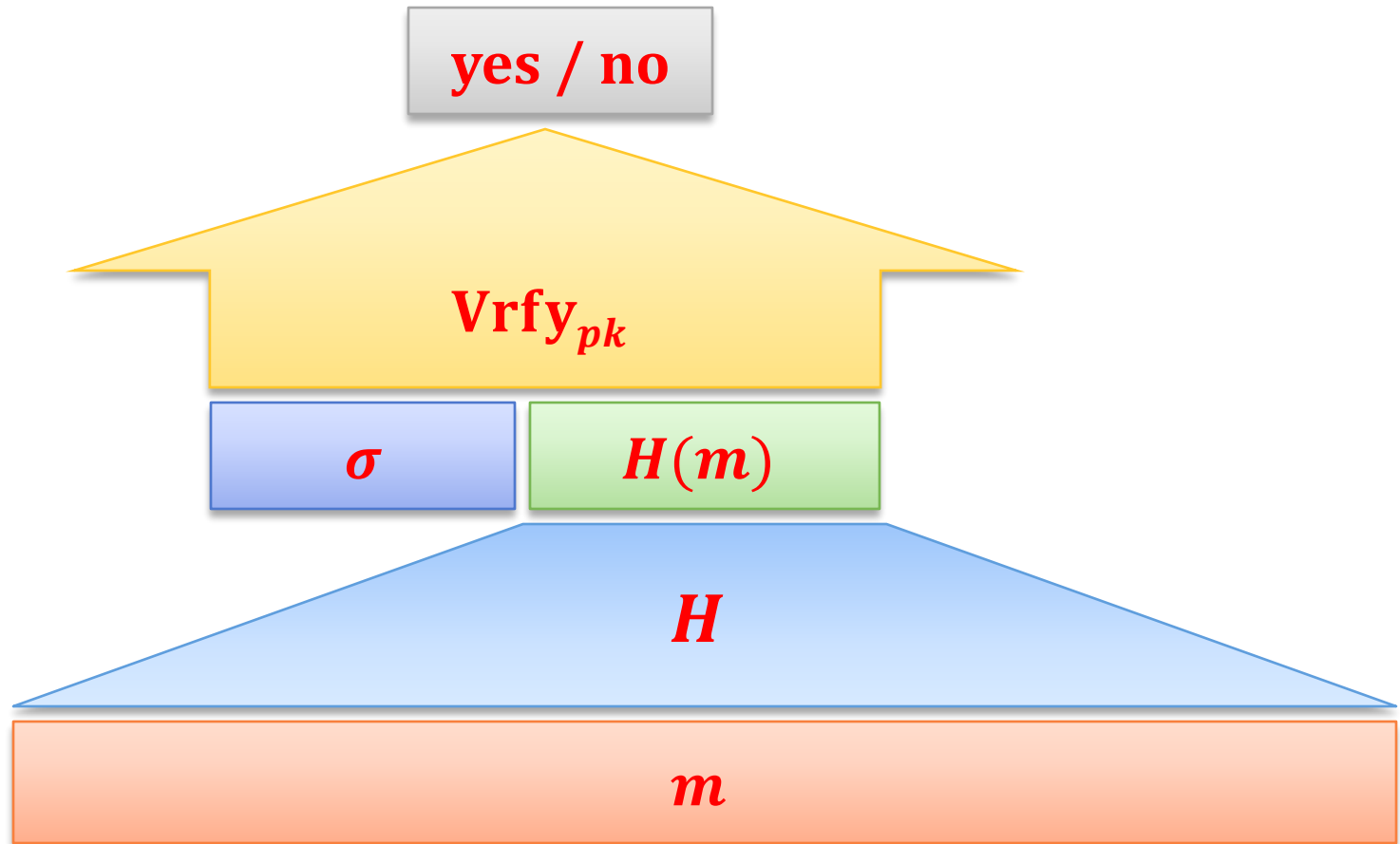
Hash-and-Sign [2/4]

How to sign a message m ?



Hash-and-Sign [3/4]

How to verify?



Hash-and-Sign [4/4]

It can be proven that this construction is secure, assuming **(Gen, Sign, Vrfy)** is secure.

For this we need to assume that **H** is taken from a family of collision-resilient hash functions.

$$\{H^s\}_{s \in \text{keys}}$$

Then **s** becomes a part of the public key and the private key.

Can anything be proven about the “hashed RSA” scheme?

In the plain model - not really.

But at least the attacks described before “look infeasible”.

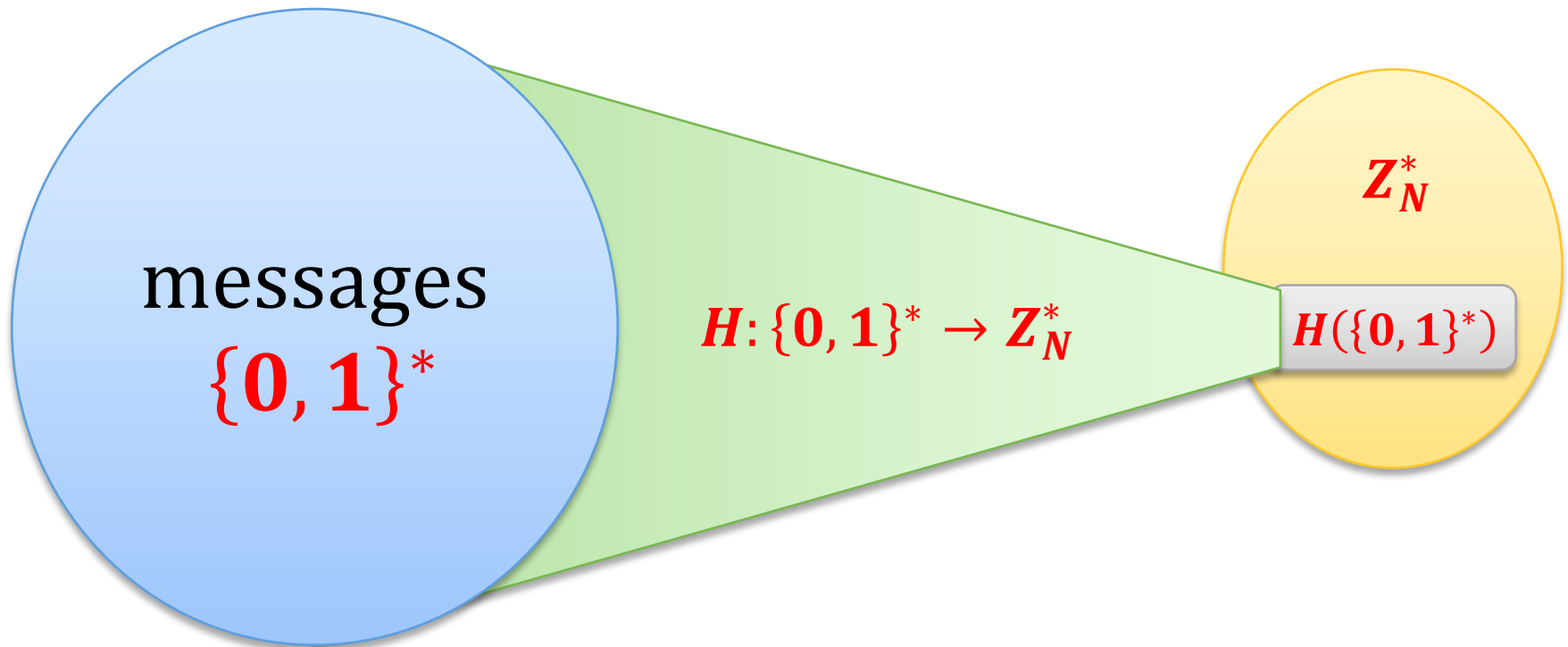
1. **For the “no message attack”:** one would need to invert **H** .
2. **The second (“homomorphic”) attack:**
Looks impossible because the adversary would need to find messages **m, m_1, m_2** such that

$$H(m) = H(m_1) \cdot H(m_2)$$

Why the security proof from the RSA assumption is impossible?

RSA assumption holds for inputs chosen **uniformly at random** from $\mathbf{Z}_N^*$.

But the output of $\mathbf{H}$ is **not** “uniformly random”



Solution: “Full Domain Hash” (FDH)

provably secure:

- under the **RSA assumption**
- and modelling ***H*** as random oracle.

Introduced in

**Bellare and Rogaway. *The exact security of digital signatures: How to sign with RSA and Rabin.*
EUROCRYPT’96**

Widely used in practice (for example in the **PKCS #1 standard**)

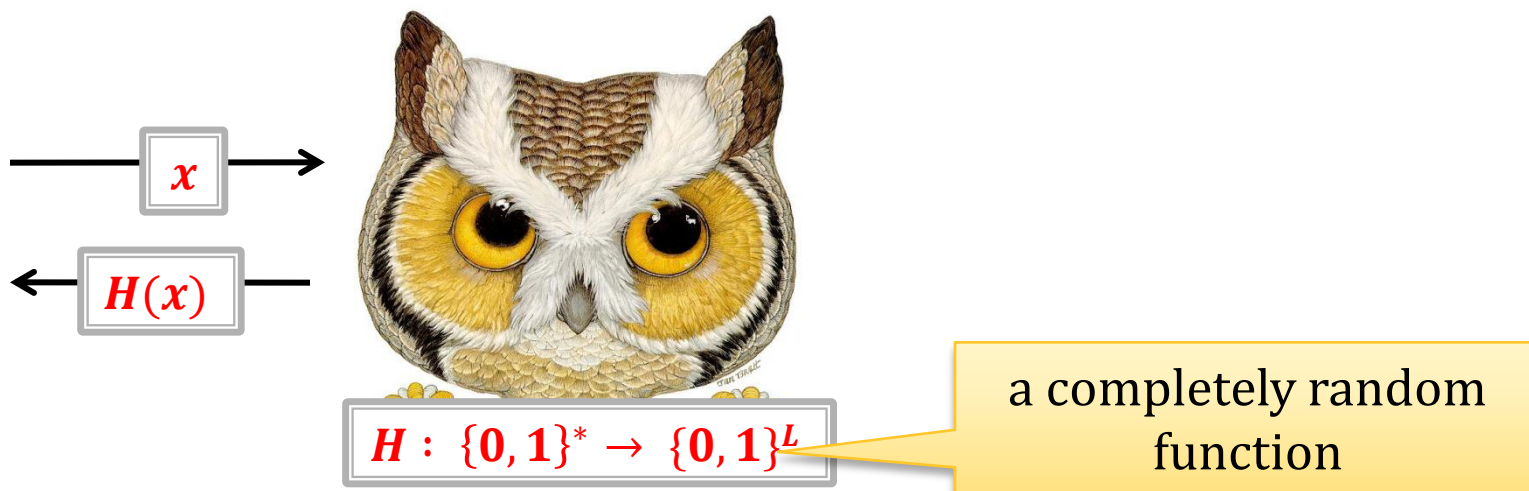
Fact (security of the **Full Domain Hash**)

Lemma (informal)

- Let $H: \{0, 1\}^* \rightarrow \mathbb{Z}_N^*$ be a hash function modeled as a **random oracle**.
- Suppose the **RSA assumption** holds

Then the “**hashed RSA**” is **existentially unforgeable under an adaptive chosen-message attack**.

Remember the Random Oracle Model?



Why does it help?

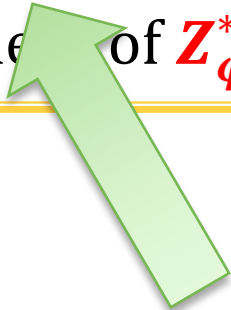
RSA assumption

For any randomized polynomial time algorithm A we have:

$$P(y^e = x \bmod N: y := A(x, N, e))$$

is negligible in $|N|$

where $N = pq$ where p and q are random primes such that $|p| = |q|$, and x is a random element of $\mathbb{Z}_N^*$, and e is a random element of $\mathbb{Z}_{\phi(N)}^*$.



here we require that x is random

Intuition

If we just use a “normal hash function” then the distribution of
 $H(m_0), H(m_1), H(m_2), \dots$
(for any $m_0, m_1, m_2, \dots$) can be “arbitrary”.

If H is a random oracle then

$H(m_0), H(m_1), H(m_2), \dots$
are uniform and independent (for pairwise different m_i 's).

This helps a lot in the proof!

Other popular signature schemes

- **Rabin** signatures (based on squaring modulo $N = pq$)

Based on discrete log (usually: in subgroups of $\mathbf{Z}_p^*$ or in elliptic curves groups):

- **ElGamal** signatures
- Digital Signature Standard (**DSS**)
- **Schnorr** signatures

can be viewed as **identification schemes** transformed using **Fiat-Shamir transform**.

we will explain it

Plan

1. The definition of secure signature schemes
2. Signatures based on RSA, “hash-and-sign”, “full-domain-hash”
3. Constructions based on discrete log
 - a) identification schemes
 - b) Schnorr signatures
 - c) DSA signatures
4. Theoretical constructions



Identification schemes

$(pk, sk) \leftarrow \text{Gen}(1^n)$ – a (public key, private key) pair of **prover**

Everybody who knows pk can verify the identity of the **prover**



prover



verifier

verifier's output:

yes if he believes he is talking to the prover

no – otherwise

Definition

We do not define identification schemes formally.

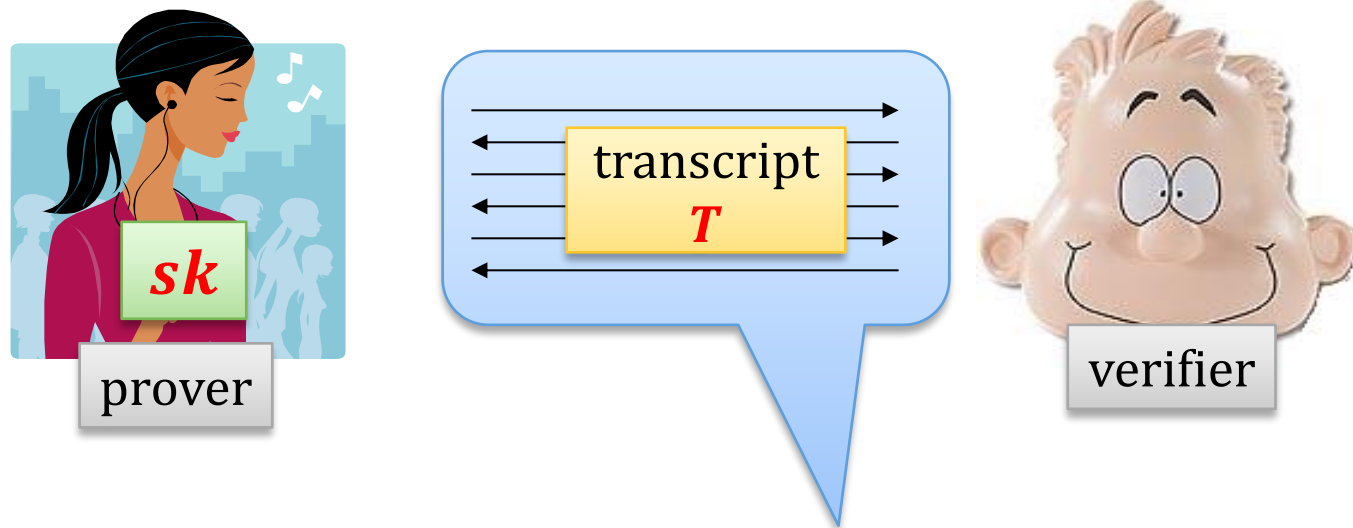
Informally they have to satisfy the following:

- **[correctness]** an honest prover should always convince the verifier
- **[security]** no poly-time adversary should be able to **impersonate the prover** with non-negligible probability.

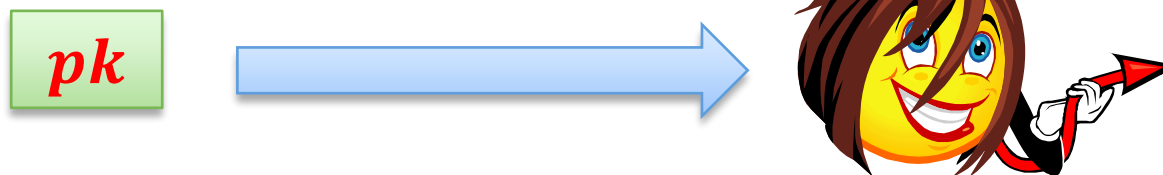
What is the attack model?

Let's assume it's rather weak:

“learning phase”:



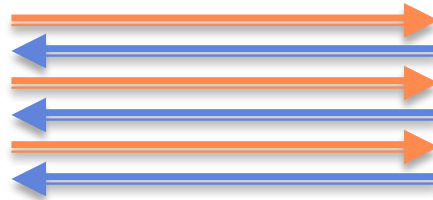
the adversary learns as many transcripts $T_1, T_2, \dots$ as she wants



Then he has to win in the following game

“challenge phase”:

the adversary does not know **sk** but has access to transcripts



the adversary wins if the verifier outputs **yes**

Note

1. The adversary **cannot talk to the prover during the learning phase.**
2. The adversary **cannot act as a man-in-the middle.**

(these problems can be solved, but are not relevant today)

We will come back again to these protocols when we talk about **zero-knowledge**.

Schnorr identification scheme

Key generation similar to the one in ElGamal encryption

Let **GenG** be such that **discrete log** is hard w. r. t. **GenG**.

Gen(1^n) first runs **GenG** to obtain G, g and q (assume q is prime). Then, it chooses $x \leftarrow \mathbb{Z}_q$ and computes $y := g^x$.

The public key is (G, g, q, y) .

The private key is (G, g, q, x) .

The protocol

G – group, $q = |G|$
 g – generator

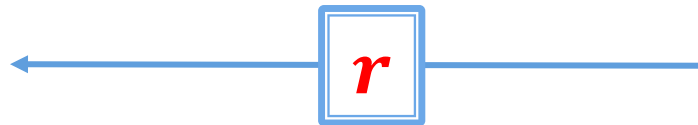
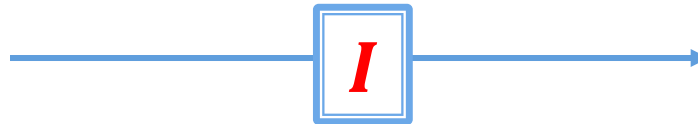


knows x



knows
 $y := g^x$

$$k \leftarrow \mathbb{Z}_q$$
$$I := g^k$$

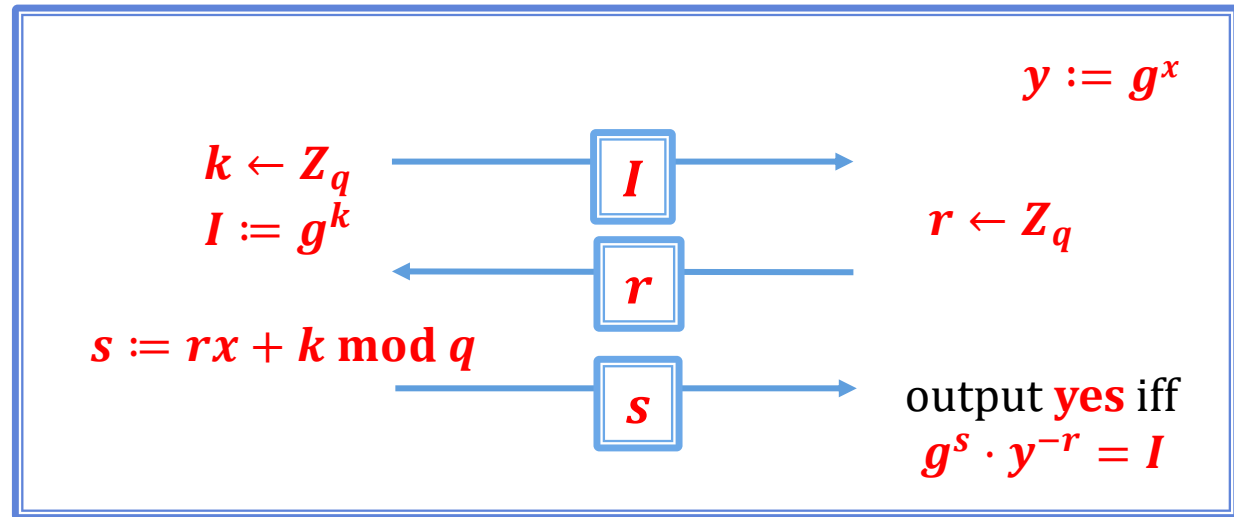


$$r \leftarrow \mathbb{Z}_q$$

$$s := rx + k \bmod q$$



output **yes** iff
 $g^s \cdot y^{-r} = I$

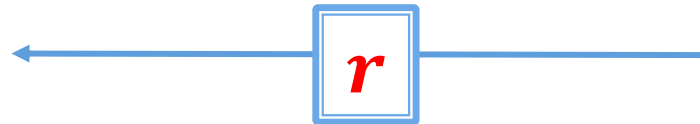
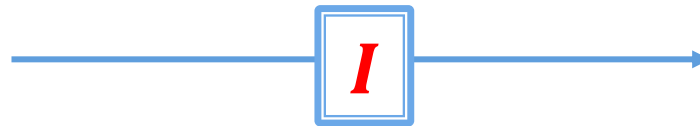


Why is this protocol correct?

$$\begin{aligned} g^s \cdot y^{-r} &= g^{rx+k} \cdot (g^x)^{-r} \\ &= g^{rx+k} \cdot g^{-rx} \\ &= g^k \\ &= I \end{aligned}$$

Security

First, suppose the adversary didn't see any transcript. He has to win the following game.



$$r \leftarrow \mathbb{Z}_q$$

output **yes** iff
 $g^s \cdot y^{-r} = I$

Lemma

If discrete logarithm is hard with respect to **GenG** then the **probability that any poly-time adversary wins this game is negligible.**

How to prove it?

We show that for every **I** there exists **at most one** **$r \in \mathbb{Z}_q$** such that **the adversary can answer it correctly** (if she cannot compute the discrete log).

(so: his probability of winning is at most **$1/q$**)

Proof by contradiction

Assume there exist r_0 and r_1 such that $r_0 \neq r_1$ and that the adversary knows answers

- s_0 to r_0 and
- s_1 to r_1

where

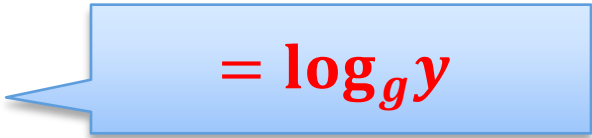
$$g^{s_0} \cdot y^{-r_0} = I = g^{s_1} \cdot y^{-r_1}.$$

But then

$$y^{r_1-r_0} = g^{s_1-s_0}$$

so

$$y = g^{\frac{s_1-s_0}{r_1-r_0}}$$


$$= \log_g y$$

This finishes the proof of the lemma.

To finish the full security proof we need to show the following

Learning the transcripts

(I, r, s)

doesn't help the adversary.

Q: Why is it true?

A: It turns out that the adversary can “simulate” such transcripts himself (just from pk).

We now explain it.

How do the transcripts look like?

$$(I, r, s)$$

where

- $I = g^k$ where $k \leftarrow \mathbb{Z}_q$
- $r \leftarrow \mathbb{Z}_q$
- $s := rx + k \bmod q$

We now show that:

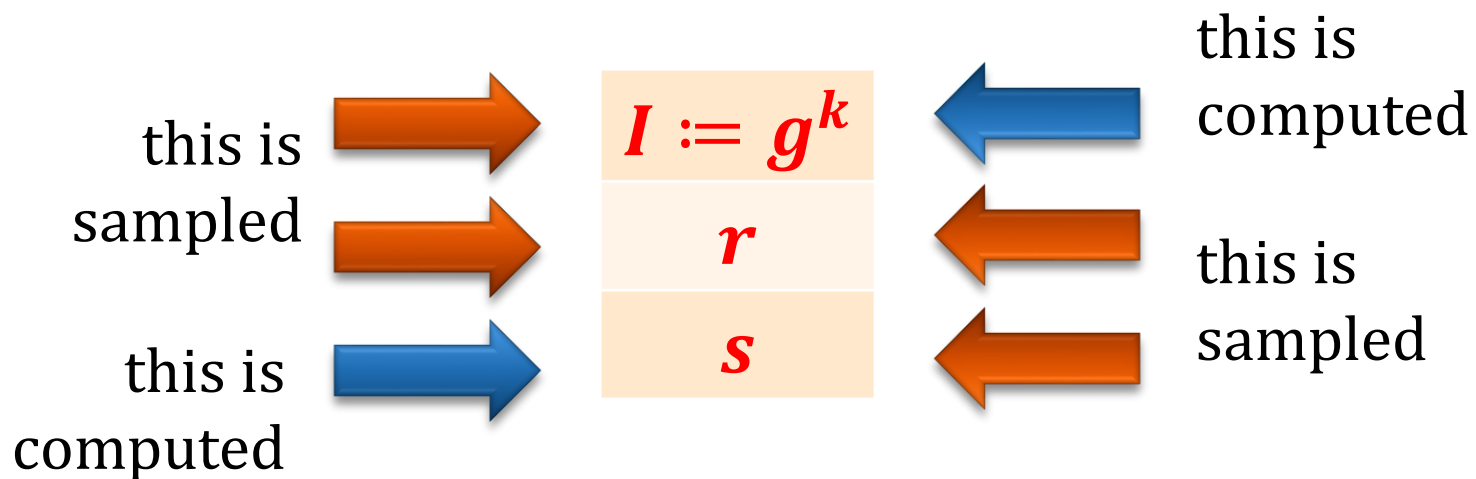
the transcripts with exactly the same distribution
can be sampled by the adversary himself!

How can the adversary do it?

- **first** sample $\mathbf{r}, \mathbf{s} \leftarrow \mathbf{Z}_q$ and
- **then** compute $\mathbf{I}$ as

$$\mathbf{I} = g^{\mathbf{s}} \cdot y^{-\mathbf{r}}$$

Why is the distribution the same?

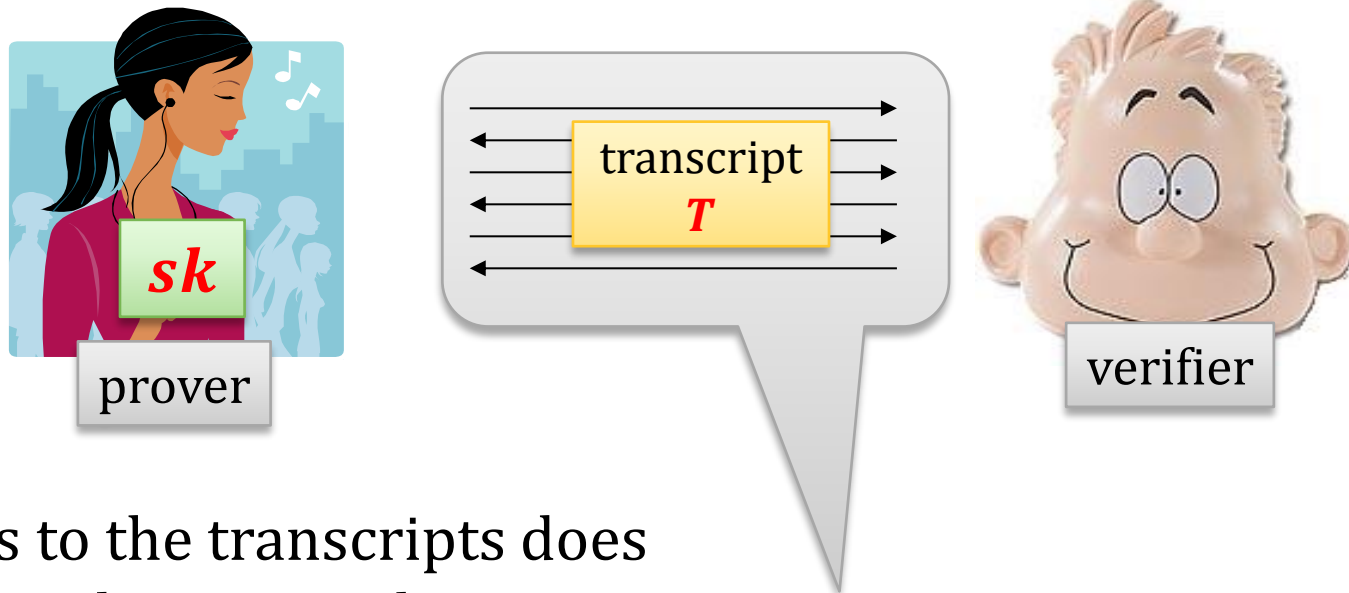


such that

$$\mathbf{s} := \mathbf{r}x + k \bmod q$$

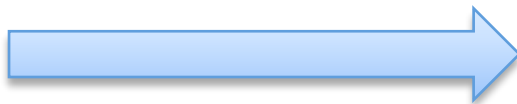
It's the same!

Therefore



access to the transcripts does
not change anything!

pk



Note the difference

The adversary **can** produce tuples (I, r, s) with the right distribution

if he “**starts from** (r, s) ”

but he **cannot do it**

if he has to “**start from** I ” and sampling r is out of his control.

Conclusion

The Schnorr protocol is a secure identification scheme.

But how is this related to the signature schemes?

We now show how to transform **any such identification scheme into a signature scheme.**

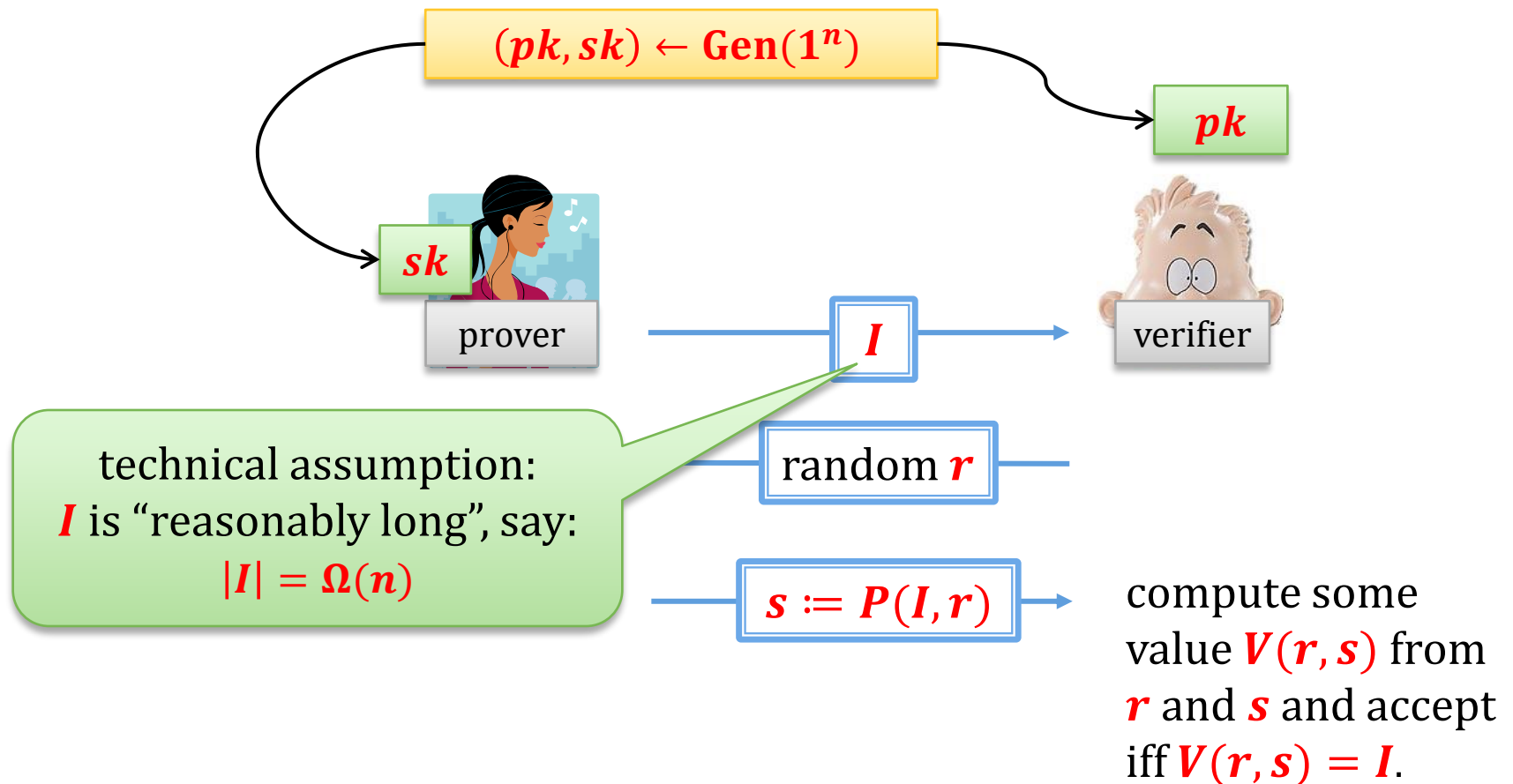
Plan

1. The definition of secure signature schemes
2. Signatures based on RSA, “hash-and-sign”, “full-domain-hash”
3. Constructions based on discrete log
 - a) identification schemes
 - b) Schnorr signatures
 - c) DSA signatures
4. Theoretical constructions



Fiat-Shamir transform: main idea

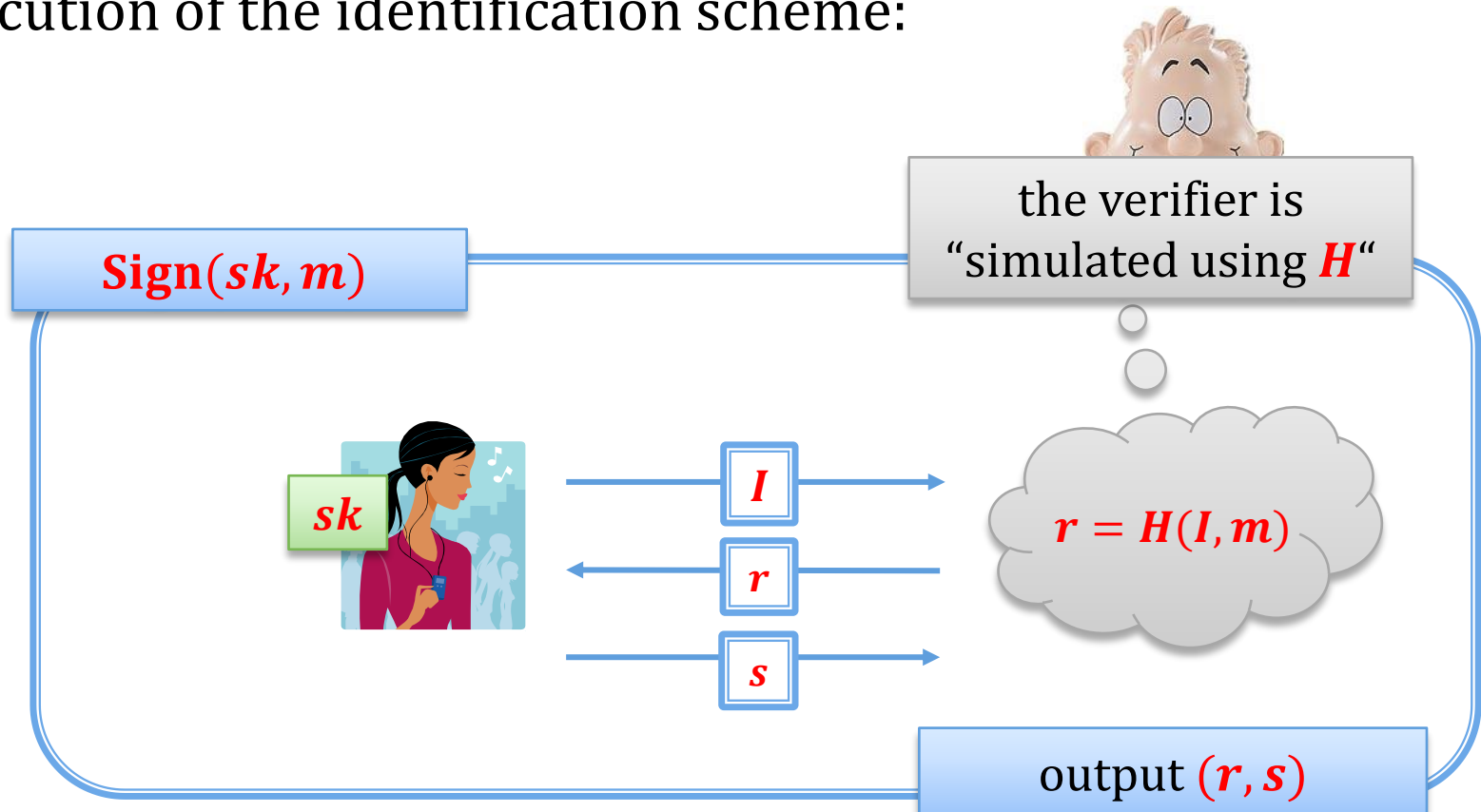
Suppose we have an identification protocol of this form:

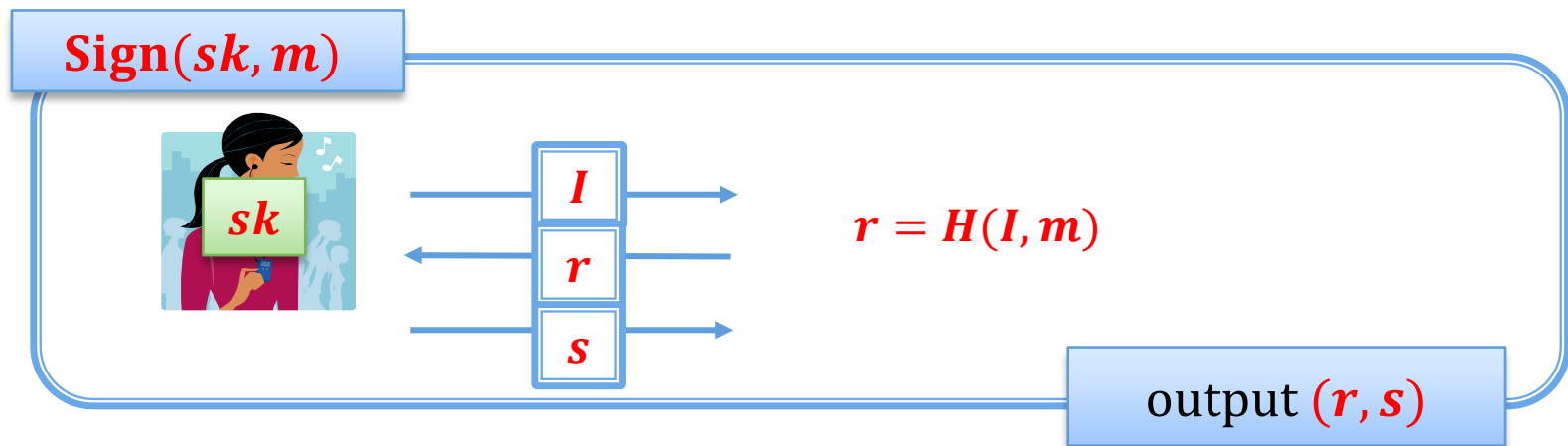


Create a signature scheme as follows

- Let $H: \{0, 1\}^* \rightarrow \{0, 1\}^{|r|}$ be a **hash function** (modelled as a **random oracle**)
- key pair generation** – as in the authentication protocol

To sign a message m the signing algorithm simulates the execution of the identification scheme:





How to verify?

Vrfy($pk, m, (r, s)$)

Assuming I is such that (I, r, s) “is a correct transcript” check if r was computed correctly. That is:

let $I = V(r, s)$
and check if $r = H(I, m)$

← equivalent →

check if $r = H(V(r, s), m)$

output **yes** iff $r = H(I, m)$

More formally

Gen – the same as in the identification scheme

Sign(sk, m) = (r, s), computed by simulating the prover or random input as follows:

1. let I be the “first message of the prover”
2. let $r := H(I, m)$
3. let $s := P(I, r)$ be the “second message of the prover” (after receiving r)
4. **output** (r, s)

Vrfy($pk, m, (r, s)$):

output yes iff $r = H(V(r, s), m)$.

Why does it work?

Correctness is trivial – if the signer is honest then the verifier will always accept.

What about **security**?

Security

First look at the **learning phase**:

In the **identification scheme** the adversary learns pk and can see many tuples of a form:

(I, r, s)

sampled as follows:

- random I
- random r
- $s := P(I, r)$

difference

In the **signature scheme** the adversary learns pk and can see many tuples of a form:

(I, r, s)

sampled as follows:

- random I
- $r := H(I, m)$
- $s := P(I, r)$

Note:

the adversary chooses m but she **cannot choose** I , so by the properties of the random oracle: r is **completely random**.

Moral:

these two experiments are identical!

Now look at the challenge phase

To break the **authentication scheme** the adversary has to find I such that after learning random r she can find s such that:

$$V(r, s) = I$$

In the **signature scheme** the adversary has to find (r, s) such that

$$r = H(I, m),$$

where $I = V(r, s)$.

since H is a random oracle

it's the same!

She has to:
choose the value of I first,
then she learns $r = H(I, m)$,
and she has to find s such

$$V(r, s) = I$$

because if he chooses r first she will not be able to find the right I (remember that I is long)

Using this method we can construct signature schemes

For example:



Schnorr's signature scheme

Gen(1^n) run **GenG** to obtain G, g and q (assume q is prime). Then, choose $x \leftarrow \mathbb{Z}_q$ and computes $y := g^x$.

- The public key pk is (G, g, q, y) .
- The private key sk is (G, g, q, x) .

Sign(sk, m):

1. choose uniform $k \leftarrow \mathbb{Z}_q$ and let $I := g^k$
2. compute $r := H(I, m)$
3. compute $s := rx + k \bmod q$
4. output (r, s)

Vrfy($pk, m, (r, s)$):

output **yes** if $r = H(g^s \cdot y^{-r}, m)$.

By the way:

It is important to check if the parameters G and g agree (“parameter validation”).
(see “[Windows CryptoAPI Spoofing Vulnerability](#)” discovered in Jan 2020 by the NSA)

Plan

1. The definition of secure signature schemes
2. Signatures based on RSA, “hash-and-sign”, “full-domain-hash”
3. Constructions based on discrete log
 - a) identification schemes
 - b) Schnorr signatures
 - c) DSA signatures
4. Theoretical constructions



DSS signatures (also called DSA)

- based on a paradigm **similar to Schnorr's signatures**
- can also be viewed as a **variant of ElGamal signatures (1984)**
- **DSS** was covered by an (expired) **U.S. Patent** 5,231,668 (**1991**) granted to the US government (available worldwide **royalty-free**)
- Schnorr claimed that his **U.S. Patent** 4,995,082 (**1989**) covered **DSA** – this claim is disputed, and anyway it expired in **2008**.
- very widely used in practice!

[we will present this scheme during the exercises]

Note

In **Schnorr** and **DSS signatures** it's very important that the signer's **randomness is generated properly** [exercise].

Failure to do so can have catastrophic effects:

BBC Sign in News Sport Weather Shop Earth Travel

NEWS

Home Video World UK Business Tech Science Magazine Entertainment & Arts

Technology

iPhone hacker publishes secret Sony PlayStation 3 key

By Jonathan Fildes
Technology reporter, BBC News

6 January 2011 | Technology [Share](#)

The PlayStation 3's security has been broken by hackers, potentially allowing anyone to run any software - including pirated games - on the console.

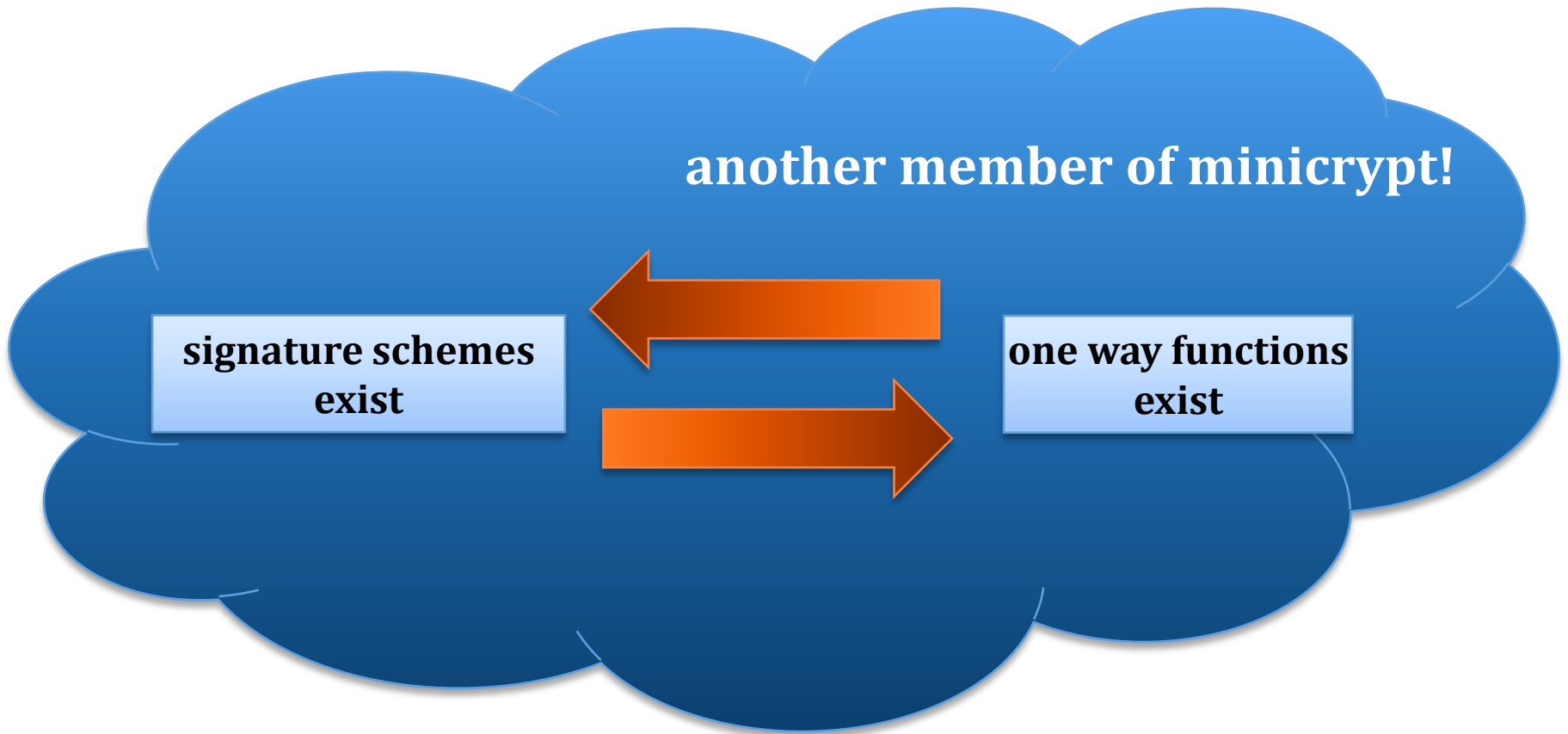


Plan

1. The definition of secure signature schemes
2. Signatures based on RSA, “hash-and-sign”, “full-domain-hash”
3. Constructions based on discrete log
 - a) identification schemes
 - b) Schnorr signatures
 - c) DSA signatures
4. Theoretical constructions



Signatures schemes can be constructed from any one-way function



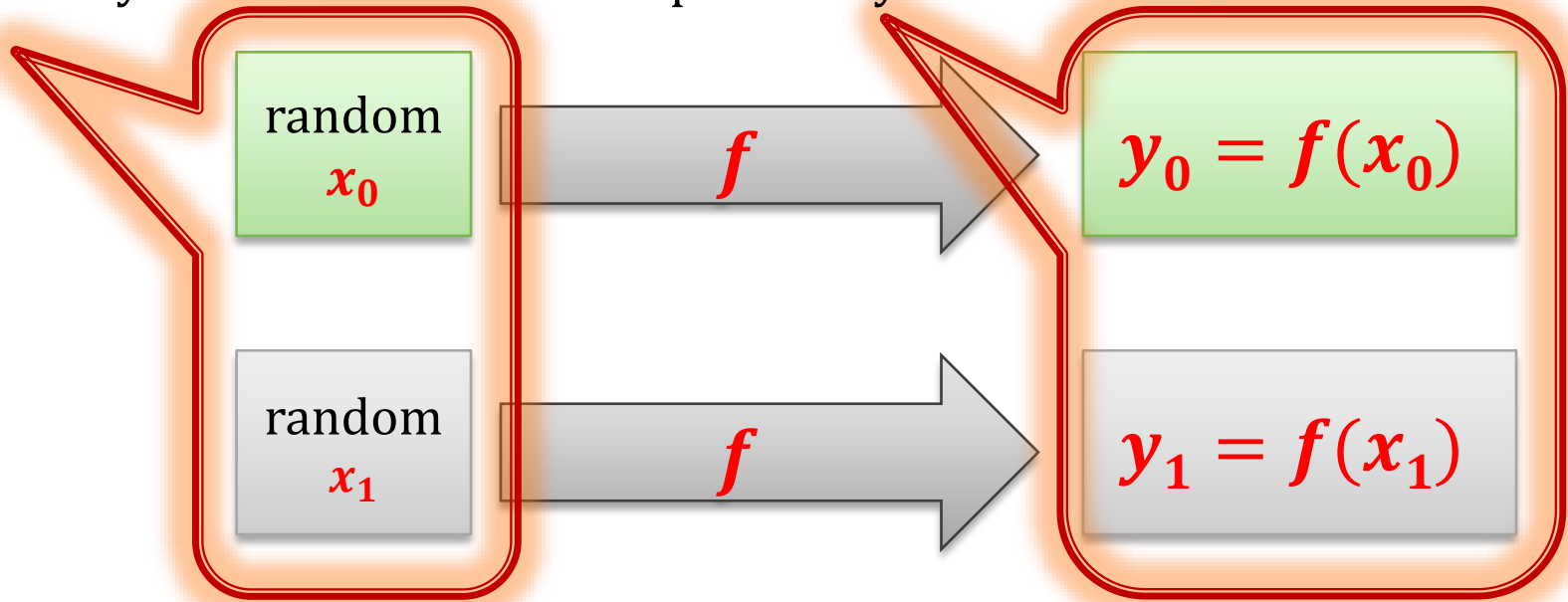
One-time signatures (Leslie Lamport)

How to sign one bit?

f – a one way function

private key

public key

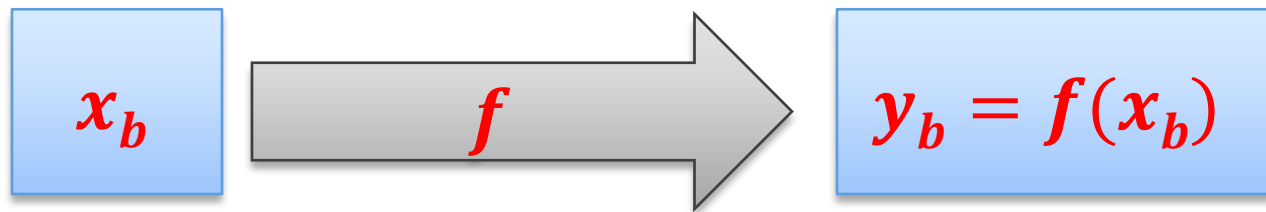


$$\text{Sign}((x_0, x_1), b) = x_b$$

$$\text{Vrfy}((y_0, y_1), b, x) = \text{yes} \text{ iff } f(x) = y_b$$

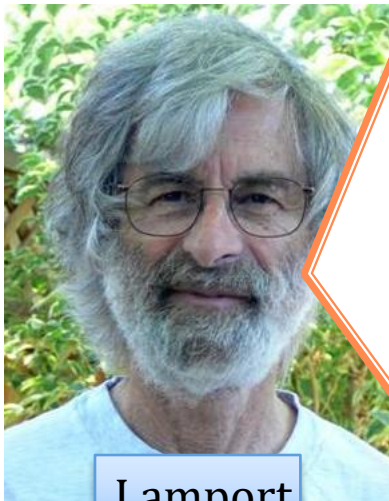
Why is it secure?

To forge a signature on bit b the adversary needs to calculate x_b from y_b



This should be infeasible, since f is one-way...

what about
the RSA ???



Lamport

Constructing Digital Signatures from a One Way Function

SRI International Technical Report CSL-98 (October 1979).

At a **coffee house in Berkeley** around **1975**, **Whitfield Diffie** described a problem to me that he had been trying to solve: constructing a digital signature for a document. **I immediately proposed a solution.** Though **not very practical**--it required perhaps 64 bits of published key to sign a single bit--it was the first digital signature algorithm.

In **1978**, **Michael Rabin** published a paper titled *Digitalized Signatures* containing a more practical scheme for generating digital signatures of documents. **(I don't remember what other digital signature algorithms had already been proposed.)** However, his solution had some drawbacks that limited its utility.

[...] I didn't feel that it added much to what Rabin had done. **However, I've been told that this paper is cited in the cryptography literature and is considered significant, so perhaps I was wrong.**

from: research.microsoft.com/en-us/um/people/lamport/

How to sign longer messages?

We show a **one-time signature** scheme (one public key can be used at most once).

f – one way function

n – length of the message

private key sk :

$x_{0,1}$	...	$x_{0,n}$
$x_{1,1}$	...	$x_{1,n}$

public key pk :

$y_{0,1} = f(x_{0,1})$	...	$y_{0,n} = f(x_{0,n})$
$y_{1,1} = f(x_{1,1})$	...	$y_{1,n} = f(x_{1,n})$

all x_{ij} 's are random strings

$$\text{Sign}_{\text{Lamport}}(sk, (m_0, \dots, m_n)) := (x_{m_0}, \dots, x_{m_n})$$

$$\text{Vrfy}_{\text{Lamport}}(pk, (m_0, \dots, m_n), (x_{m_0}, \dots, x_{m_n})) :=$$

check if $(f(x_{m_0}), \dots, f(x_{m_n})) = (y_{m_0}, \dots, y_{m_n})$

Example

$$n = 6$$

$$m = (1, 0, 1, 1, 0, 0)$$

private key:

$x_{0,1}$	$x_{0,2}$	$x_{0,3}$	$x_{0,4}$	$x_{0,5}$	$x_{0,6}$
$x_{1,1}$	$x_{1,2}$	$x_{1,3}$	$x_{1,4}$	$x_{1,5}$	$x_{1,6}$

public key:

$f(x_{0,1})$	$f(x_{0,2})$	$f(x_{0,3})$	$f(x_{0,4})$	$f(x_{0,5})$	$f(x_{0,6})$
$f(x_{1,1})$	$f(x_{1,2})$	$f(x_{1,3})$	$f(x_{1,4})$	$f(x_{1,5})$	$f(x_{1,6})$

signature:

$x_{1,1}$ $x_{0,2}$ $x_{1,3}$ $x_{1,4}$ $x_{0,5}$ $x_{0,6}$

Why each key can be used at most once?

private key:

$x_{0,1}$	$x_{0,2}$	$x_{0,3}$	$x_{0,4}$	$x_{0,5}$	$x_{0,6}$
$x_{1,1}$	$x_{1,2}$	$x_{1,3}$	$x_{1,4}$	$x_{1,5}$	$x_{1,6}$

signature:

$x_{1,1}$ $x_{0,2}$ $x_{1,3}$ $x_{1,4}$ $x_{0,5}$ $x_{0,6}$

private key:

$x_{0,1}$	$x_{0,2}$	$x_{0,3}$	$x_{0,4}$	$x_{0,5}$	$x_{0,6}$
$x_{1,1}$	$x_{1,2}$	$x_{1,3}$	$x_{1,4}$	$x_{1,5}$	$x_{1,6}$

signature:

$x_{1,1}$ $x_{1,2}$ $x_{1,3}$ $x_{0,4}$ $x_{1,5}$ $x_{1,6}$

private key:

$x_{0,1}$	$x_{0,2}$	$x_{0,3}$	$x_{0,4}$	$x_{0,5}$	$x_{0,6}$
$x_{1,1}$	$x_{1,2}$	$x_{1,3}$	$x_{1,4}$	$x_{1,5}$	$x_{1,6}$

signature:

$x_{1,1}$ $x_{0,2}$ $x_{1,3}$ $x_{0,4}$ $x_{1,5}$ $x_{1,6}$

knows



can
calculate

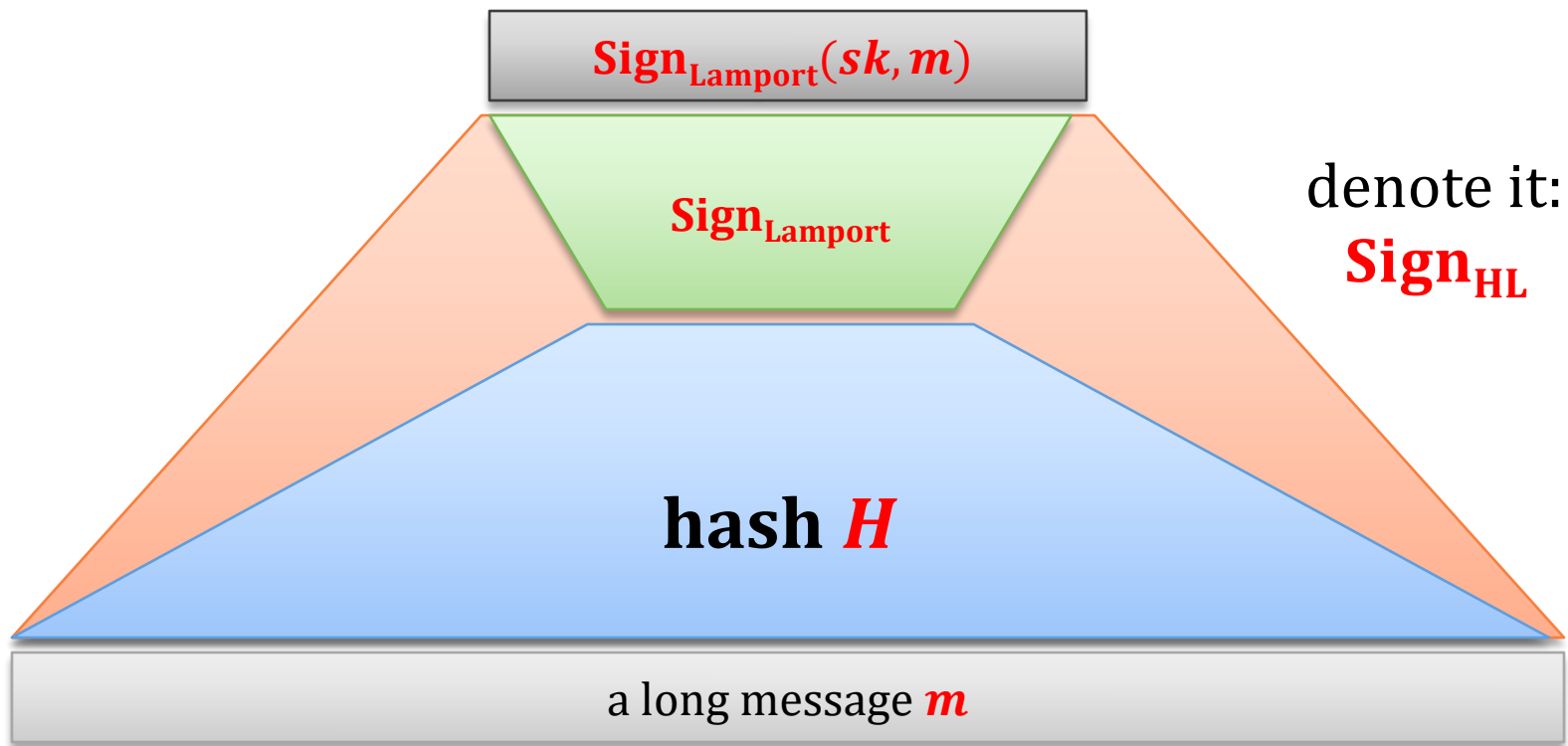
Problem

Signature is much **longer than the message!**

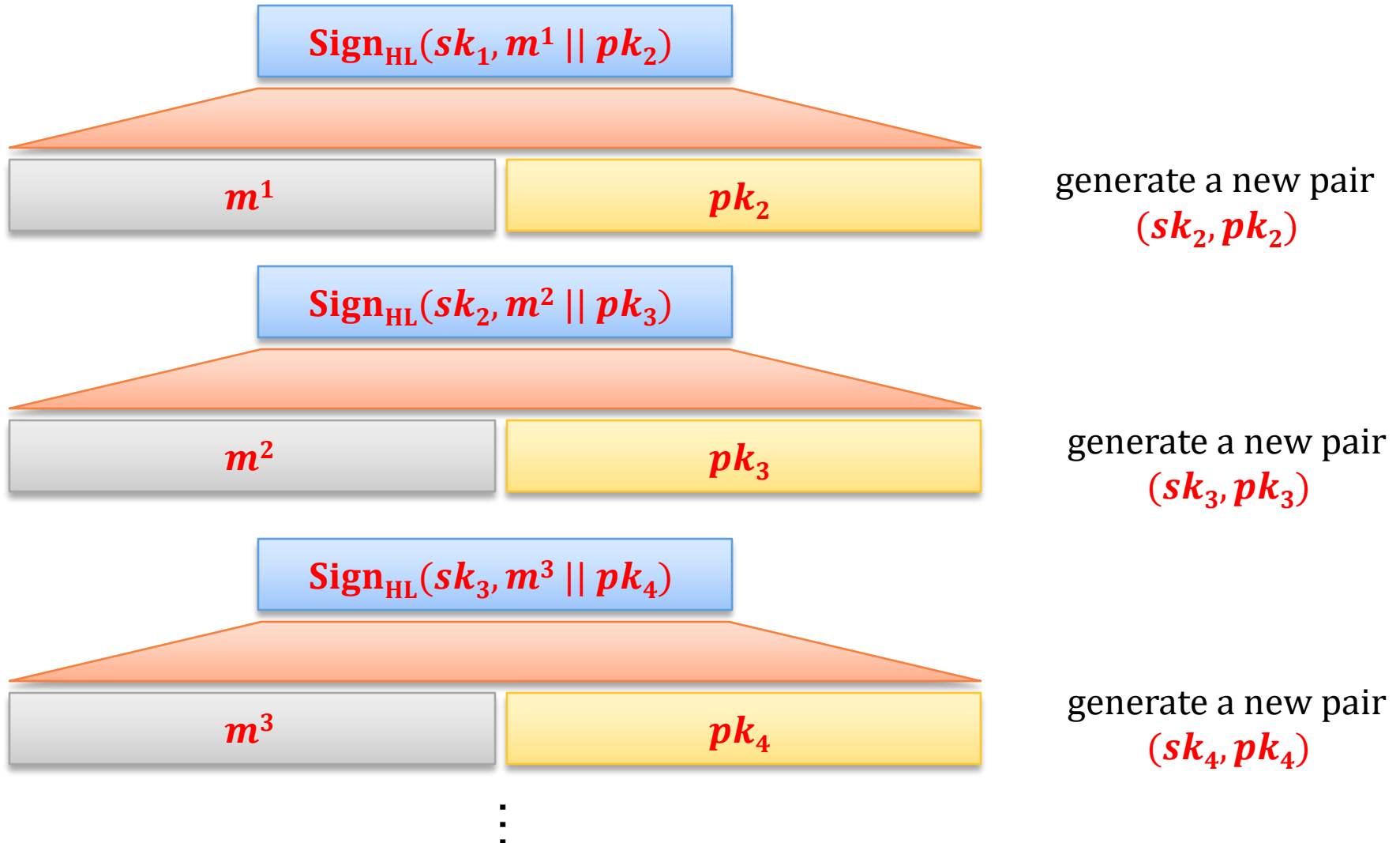
(and can be used **only once**)

How to sign long messages?

Use hash functions

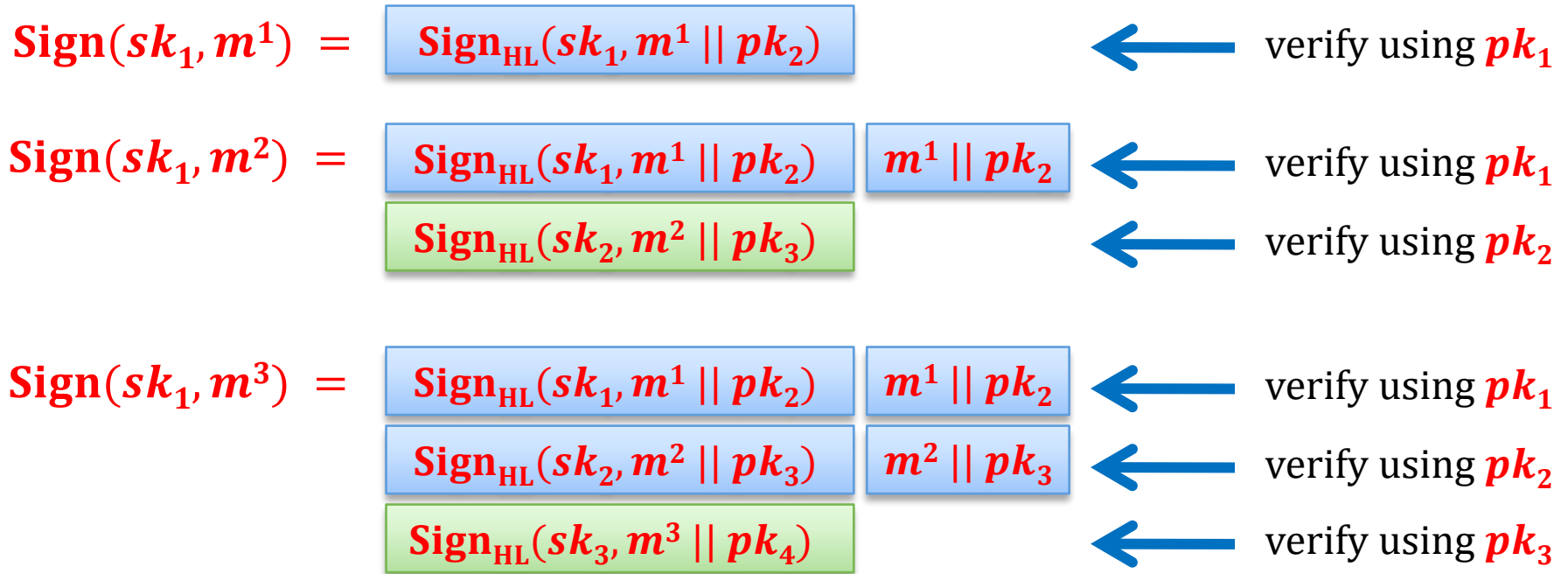


Idea: to sign multiple messages use “certification”



How to verify?

The signer needs to include the “certificate chain” in the signature.



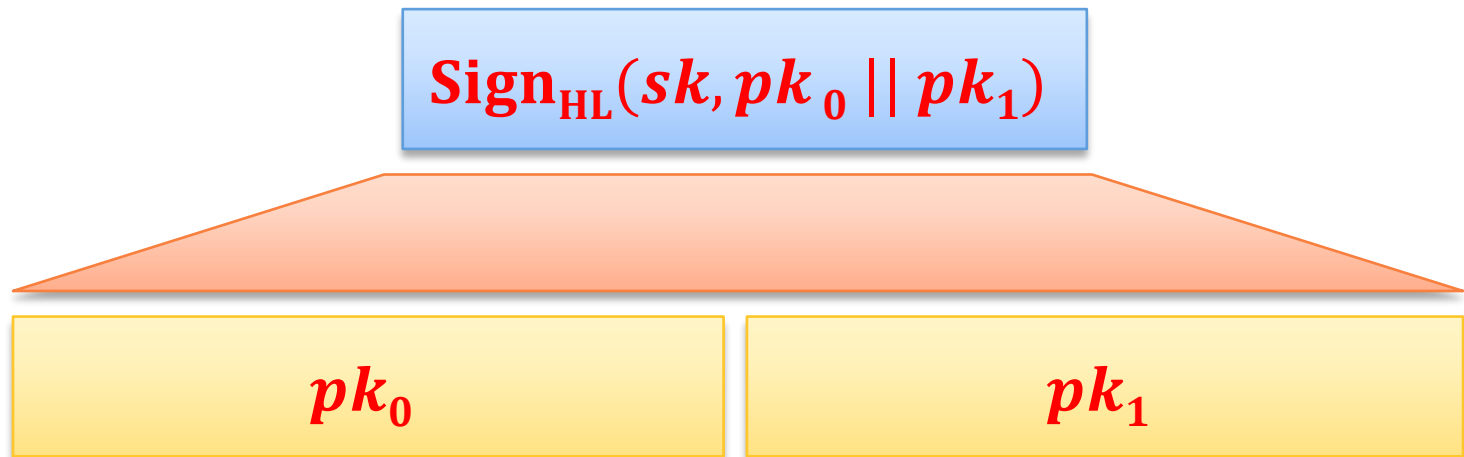
Problems

1. The length of the signature **grows linearly**
2. The signing algorithm needs to **have a state** (“memory”)

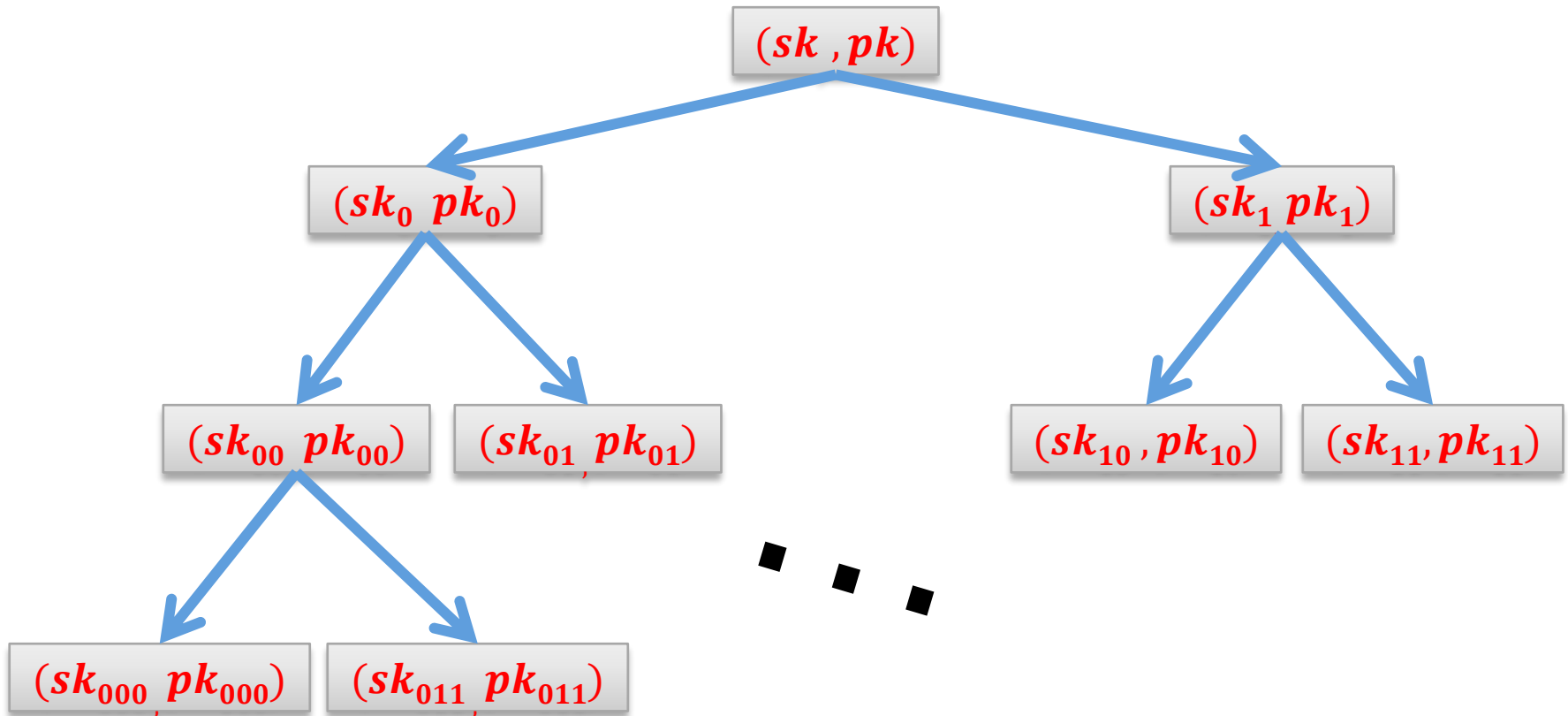
Solution to the first problem

Instead of a **chain** use a **binary tree**:

“certify each time **two** public keys”



The tree:



The details

$$m = (m_1, \dots, m_t)$$

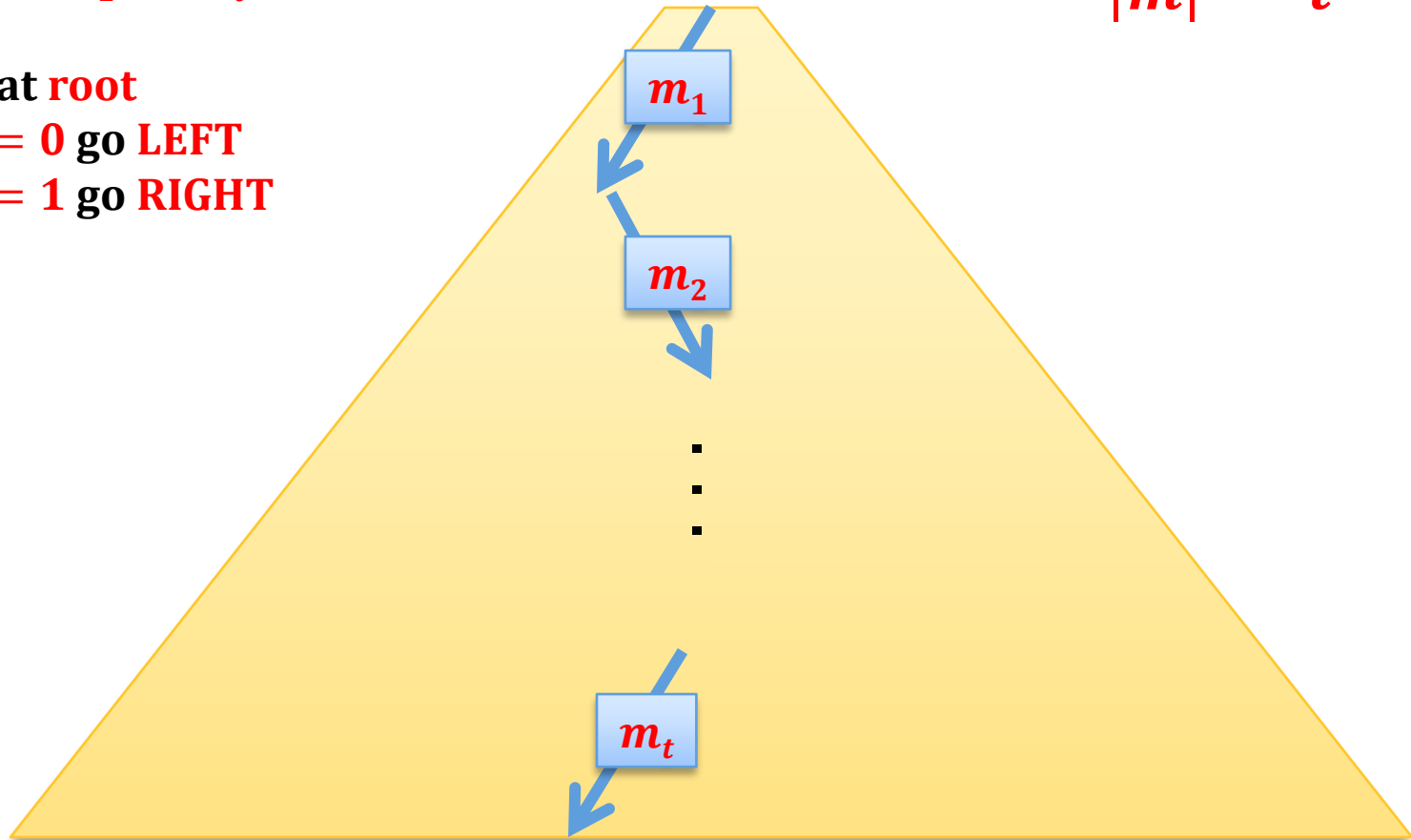
start at **root**

if $m_i = 0$ go **LEFT**

if $m_i = 1$ go **RIGHT**

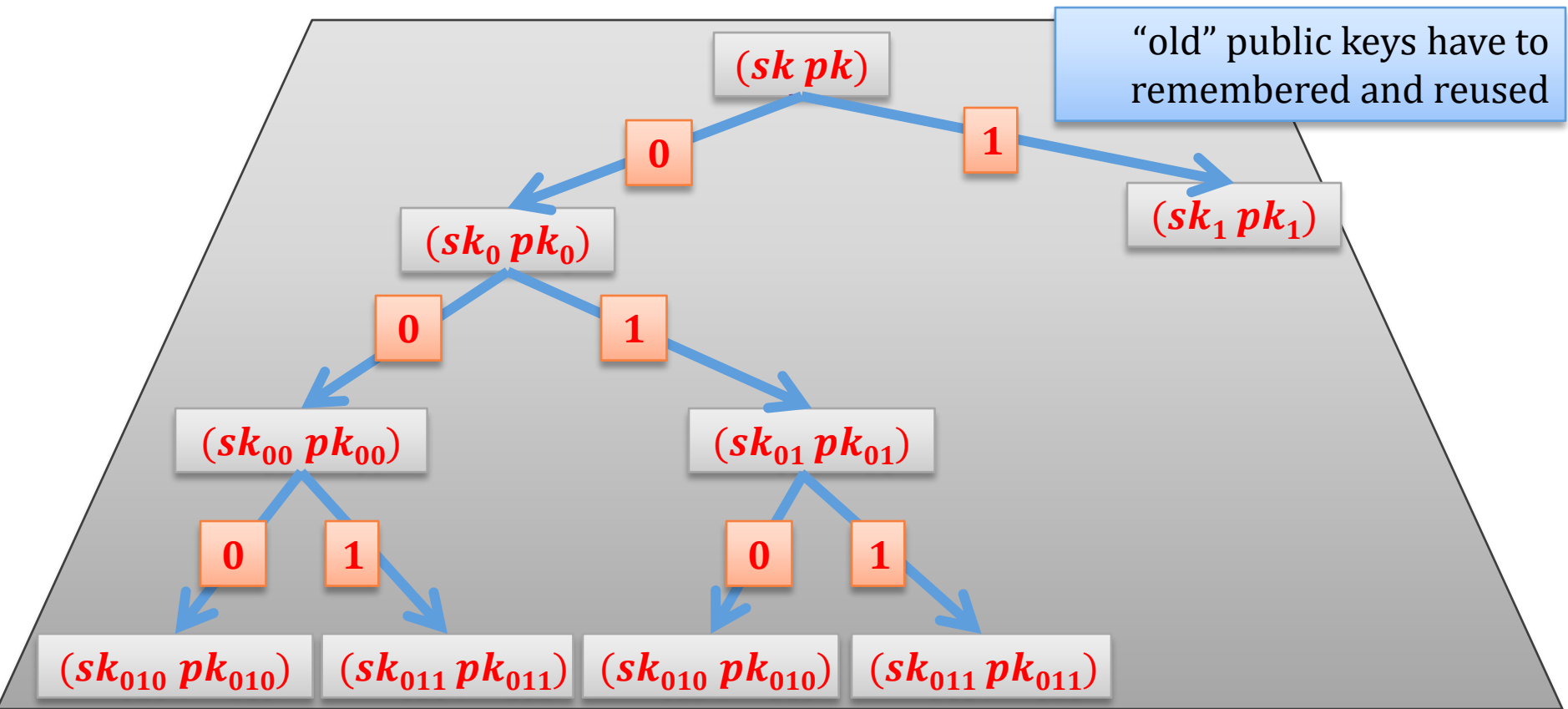
now the “chain” has length

$$|m| = t$$



use the key in the **LEAF** to sign m

The key pairs are generated on-fly



$$\text{Sign}(sk, (0, 1, 0)) =$$

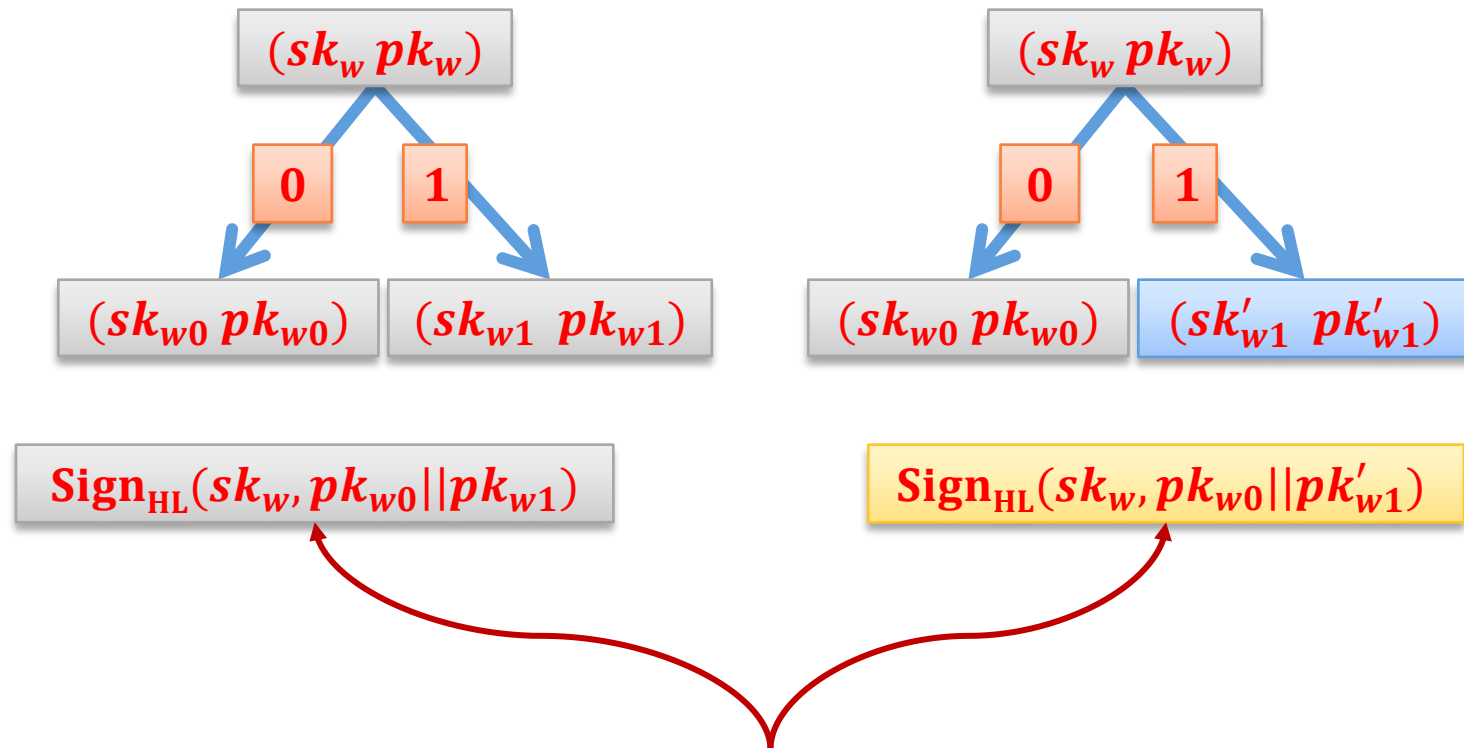
$$\text{Sign}_{\text{HL}}(sk, pk_0 || pk_1) \quad \text{Sign}_{\text{HL}}(sk_0, pk_{00} || pk_{01}) \quad \text{Sign}_{\text{HL}}(sk_{01}, pk_{010} || pk_{011}) \quad \text{Sign}_{\text{HL}}(sk_{010}, (0, 1, 0))$$

$$\text{Sign}(sk, (0, 0, 0)) =$$

$$\text{Sign}_{\text{HL}}(sk, pk_0 || pk_1) \quad \text{Sign}_{\text{HL}}(sk_0, pk_{00} || pk_{01}) \quad \text{Sign}_{\text{HL}}(sk_{00}, pk_{000} || pk_{001}) \quad \text{Sign}_{\text{HL}}(sk_{000}, (0, 0, 0))$$

Why we have to remember the old keys?

Suppose we don't:



so we signed two different messages with the same key!

Problem

The tree is constructed on-fly, so we need to remember the state.

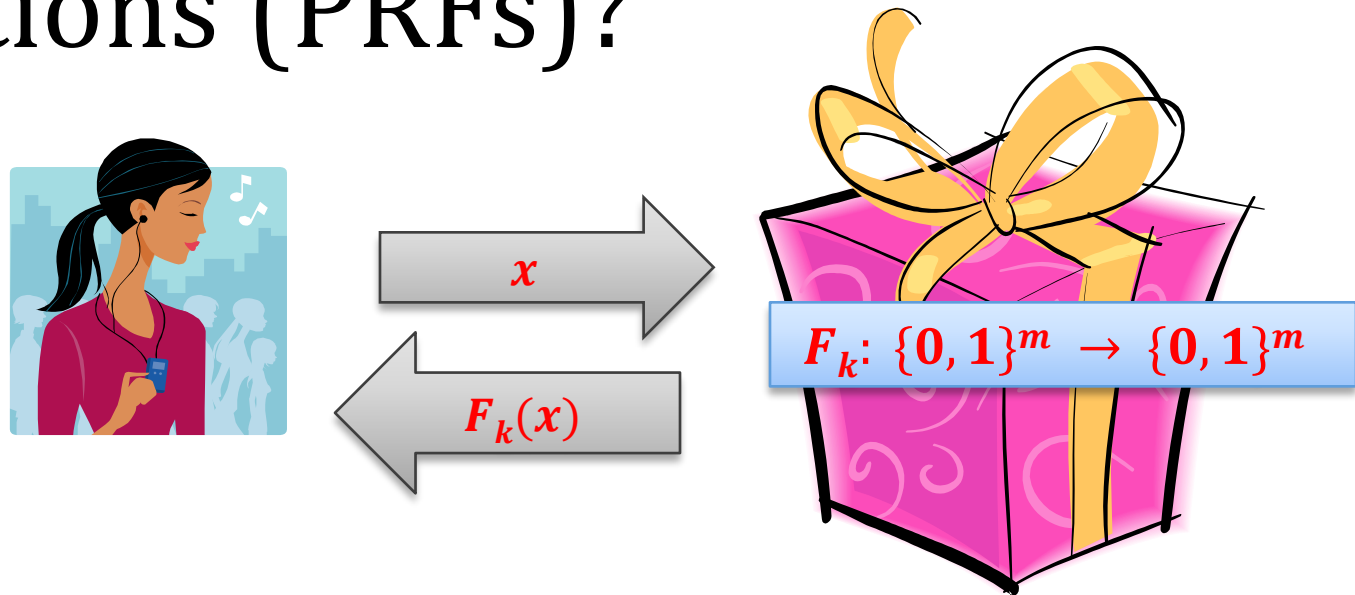
A stupid solution:

generate the whole tree beforehand.

A better solution:

generate the whole tree pseudorandomly and just remember the seed.

Remember the pseudorandom functions (PRFs)?



For a random key k and any $x_1, \dots, x_t$ the values $F_k(x_1), \dots, F_k(x_t)$ “look random”

Solution

Take some PRF F

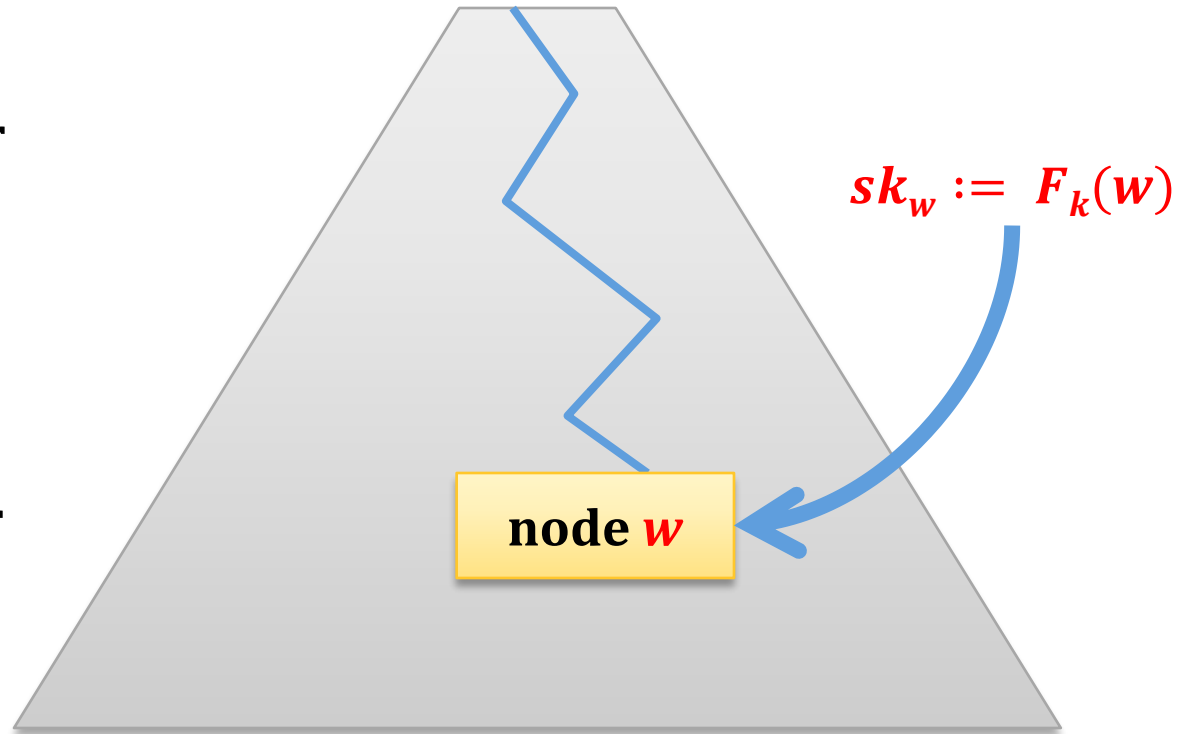
private key: (sk, k)

sk – a private key for
hashed Lamport

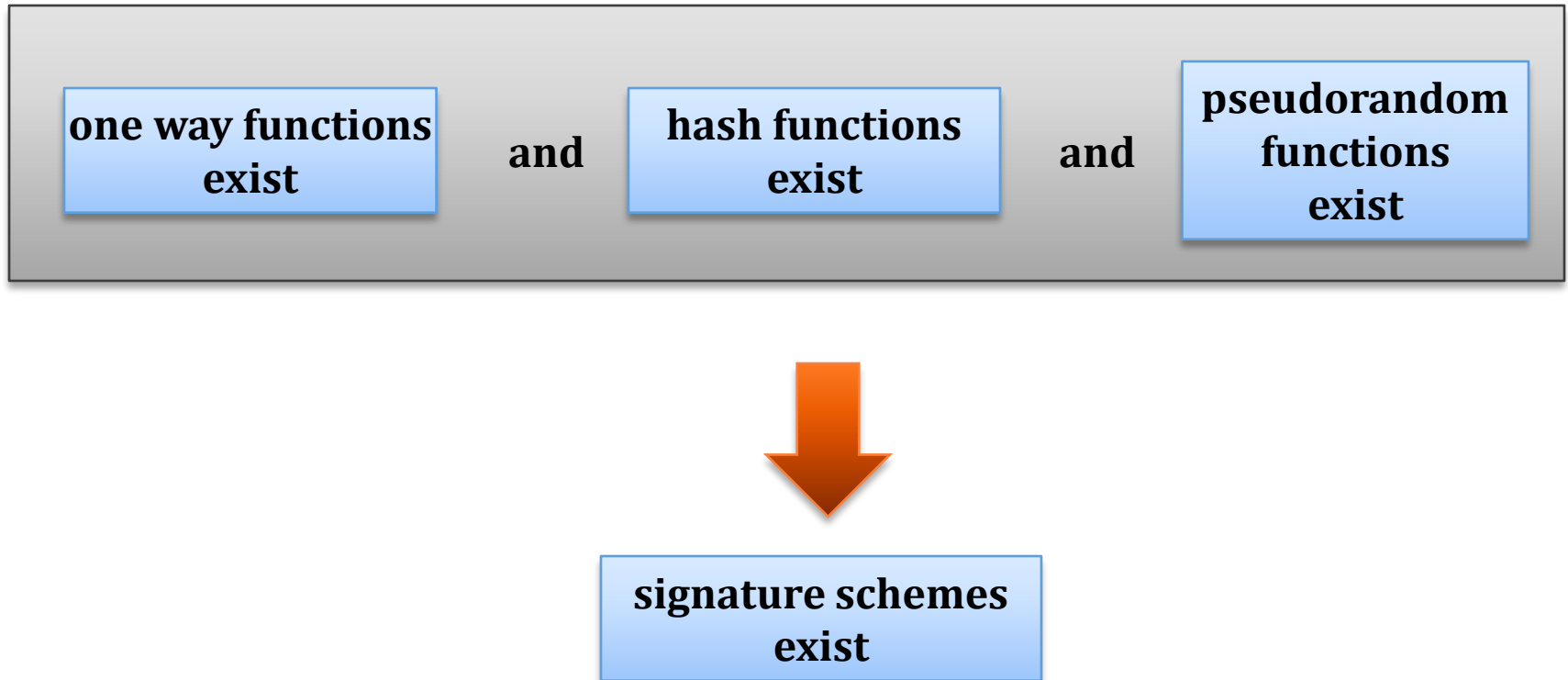
k – a key for PRF F

public key:

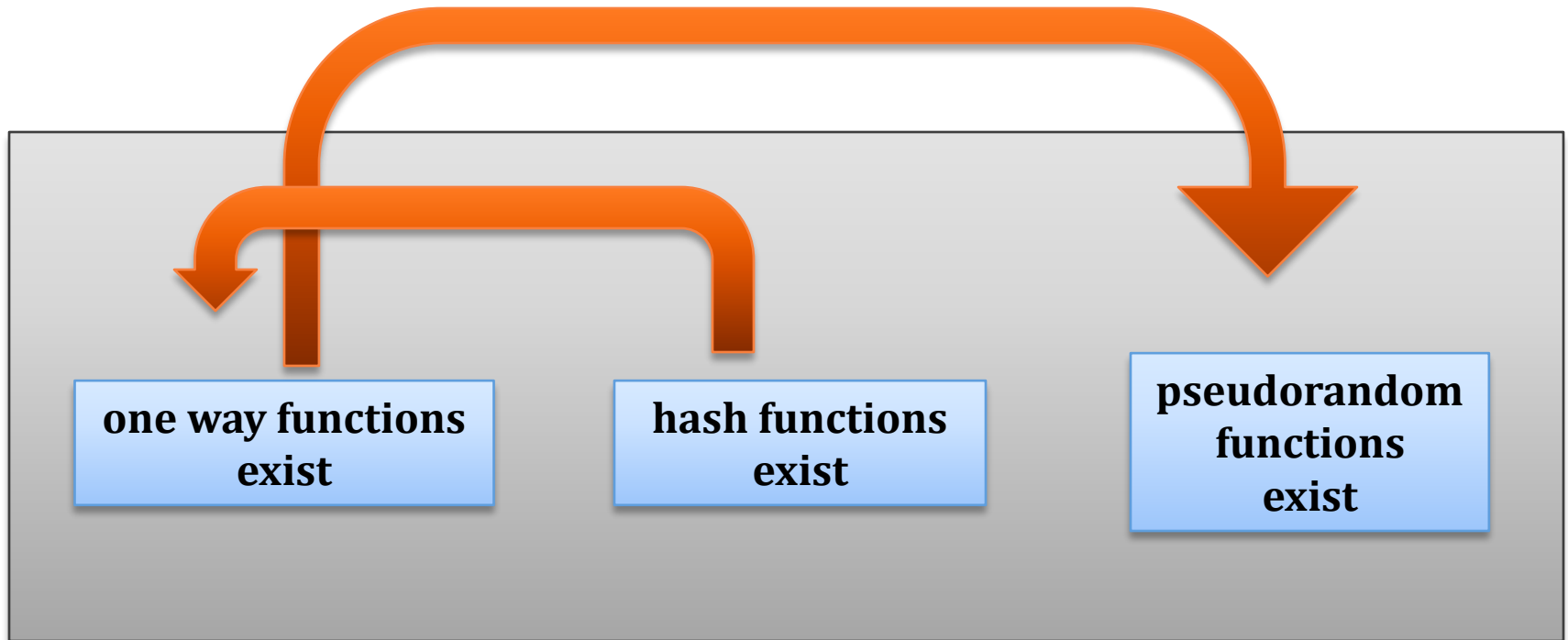
pk – a public key for
hashed Lamport



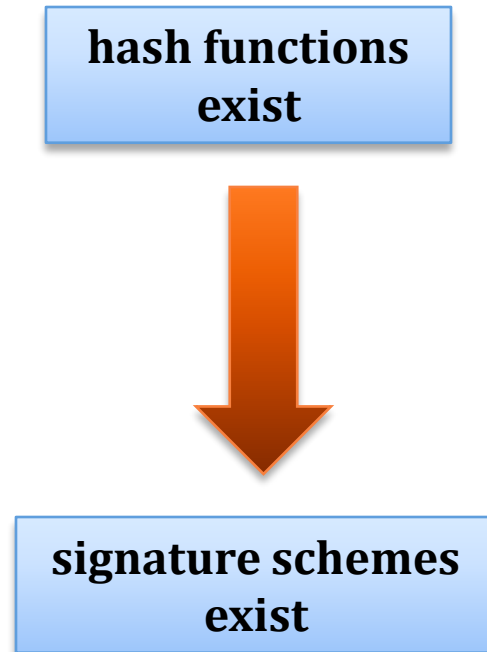
We have shown that



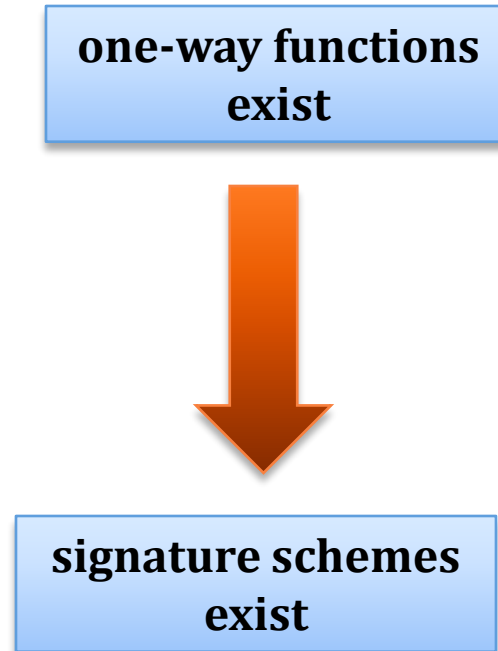
But we know that



Therefore we have shown that



The proof that



is more complicated

©2025 by Stefan Dziembowski. Permission to make digital or hard copies of part or all of this material is currently granted without fee *provided that copies are made only for personal or classroom use, are not distributed for profit or commercial advantage, and that new copies bear this notice and the full citation.*